

DHRL-FNMR: An Intelligent Multicast Routing Approach Based on Deep Hierarchical Reinforcement Learning in SDN

Miao Ye[✉], Chenwei Zhao, Peng Wen, Yong Wang[✉], Xiaoli Wang[✉], and Hongbing Qiu[✉]

Abstract—The multicast routing problem in software-defined networking (SDN) is an NP-hard problem. The existing solution methods based on deep strength learning suffer from the problems of branch redundancy, an excessively large action space and slow convergence of the intelligent models. In this paper, an intelligent multicast routing algorithm based on deep hierarchical reinforcement learning is proposed to circumvent the aforementioned problems. First, the optimal multicast tree problem is decomposed into two subproblems: fork node selection and the construction of an optimal path from a fork node to a destination node. Second, a multichannel matrix is designed as the state space for the internal and external controllers of hierarchical reinforcement learning based on the global network-aware information characteristics of SDN. Then, different action spaces are designed for the upper and lower subproblems, four action selection policies are designed for constructing multicast paths, and different reward policies are designed at different levels. Finally, a series of experiments and their results show that the designed algorithm not only searches the multicast tree efficiently but also converges faster and without redundant branches, with better performance in terms of bandwidth, delay and packet loss rate than the current mainstream solution algorithms. The codes for DHRL-FNMR are open and available at <https://github.com/GuetYe/DHRL-FNMR>.

Index Terms—Deep hierarchical reinforcement learning, multicast tree, deep reinforcement learning, software-defined networking.

Manuscript received 7 February 2024; accepted 28 April 2024. Date of publication 20 May 2024; date of current version 16 October 2024. This work was supported in part by National Natural Science Foundation of China (Nos. 62161006, 62372353, U22A2098), the subsidization of Innovation Project of Guangxi Graduate Education (No. YCBZ2023134), Key Laboratory of Cognitive Radio and Information Processing, Ministry of Education (Guilin University of Electronic Technology) (No. CRKL220103), and Guangxi Key Laboratory of Wireless Wideband Communication and Signal Processing (Nos. GXKL06230102). The associate editor coordinating the review of this article and approving it for publication was R. Pasquini. (Corresponding author: Xiaoli Wang.)

Miao Ye is with the School of Information and Communication, Guilin University of Electronic Technology, Guilin 541004, China, and also with the School of Computer Science and Technology, Xidian University, Xi'an 710071, Shaanxi, China (e-mail: ym@mail.xidian.edu.cn).

Chenwei Zhao, Peng Wen, and Hongbing Qiu are with the School of Information and Communication, Guilin University of Electronic Technology, Guilin 541004, China (e-mail: acesohn@mail.guet.edu.cn; wenpeng1994325@gmail.com; qiuhb@guet.edu.cn).

Yong Wang is with the School of Computer and Information Security, Guilin University of Electronic Technology, Guilin 541004, China (e-mail: ywang@guet.edu.cn).

Xiaoli Wang is with the School of Computer Science and Technology, Xidian University, Xi'an 710071, Shaanxi, China (e-mail: wangxiaoli@mail.xidian.edu.cn).

Digital Object Identifier 10.1109/TNSM.2024.3402275

I. INTRODUCTION

THE INCREASING prevalence of “one-to-many” multicast data transmission in network applications is a result of the rapid development of network infrastructure. By transmitting multicast data only once along the same path instead of multiple times, it is possible to reduce network traffic and congestion. To achieve this, it is necessary to find suitable fork nodes for constructing an optimal multicast tree and transmitting multicast data along that tree.

The problem of constructing the optimal multicast tree can be modeled as a classical discrete optimization problem in mathematical graph theory—the Steiner tree problem. However, dynamic changes in the network link state at different times pose challenges in optimal multicast tree construction. Therefore, timely and convenient collection and acquisition of network state information, as well as the design of algorithms for constructing optimal multicast trees, are indispensable for the multicast routing of data. Compared with traditional network measurement methods for obtaining network state information, software-defined networking (SDN) technology, which is based on a structure with separation between the data plane and the control plane, can effectively solve the problems of limited network state information, poor adaptability to dynamic and complex network changes, and inflexible data forwarding. By communicating with the data plane through the southbound interface of the SDN controller, centralized control and easy configuration and management of the network can be achieved, and data-plane SDN switches can be used for state collection and forwarding routing control by means of flow table distribution. The centralized control programming method of the SDN controller provides a possibility for the implementation of multicast routing algorithms. Therefore, this article mainly focuses on the design and implementation of SDN multicast routing algorithms for dynamically changing networks under the SDN architecture.

The Steiner tree problem, which corresponds to the multicast tree problem, is an NP-complete problem [1]. Classical algorithms such as the KMB algorithm (proposed by Kou, Markowsky, and Berman) [2], the Minimum-cost Path Heuristic (MPH) algorithm [3], and the average distance heuristic (ADH) can approximate Steiner trees. However, they fail to account for global dynamic changes in the network and have a weaker ability to adapt to dynamic changes in the network. Intelligent optimization algorithms such as

ant colony optimization (ACO) [4] and genetic algorithms (GAs) [5] suffer from high time consumption and difficulty in convergence for global optimization. Therefore, alternative methods are required to solve the multicast tree problem effectively.

Reinforcement learning (RL) methods aim to maximize rewards through agent–environment interaction. Offline learning can be applied to train an agent without online computation, thereby reducing computational time costs. RL has yielded improvements in routing path construction [6], [7], but current RL methods in multicast research consider only single factors, such as hop count, and rarely consider multiple factors. In our previous work, the DRL-M4MR algorithm [8] was proposed to address this issue; however, this algorithm considers all links as the action space. This makes it easy for the agent to select negative samples during the early stages of learning, leading to slow convergence. Additionally, redundant branches can occur during multicast tree construction.

To address the issues previously mentioned, this paper proposes an intelligent SDN multicast routing algorithm called DHRL-FNMR (finding Fork Nodes to construct Multicast Routing paths based on Deep Hierarchical Reinforcement Learning) based on deep hierarchical reinforcement learning (DHRL). We model the process of multicast tree construction as a semi-Markov decision process (SMDP). By performing multiple actions in a single execution, the agent can reach states in more distant time steps through an SMDP [9]. Accordingly, the process of multicast tree construction can be divided into two subproblems. The first is the upper-level subproblem of selecting a fork node, while the second is the lower-level subproblem of constructing the optimal path from a fork node to a destination node. Once the intrinsic controller and meta-controller have built optimal unicast paths from fork nodes to destination nodes multiple times, the agent can construct the optimal multicast tree.

In this paper, the global network awareness of SDN is used to enable the learning agent to perceive global network information. A multicast tree state matrix, a link bandwidth matrix, a link delay matrix, a link packet loss rate matrix, and a subgoal matrix are designed to form the state spaces for the intrinsic controller and meta-controller. To address the issue of an excessively large action space for the agent, different action spaces are designed for the upper- and lower-level subproblems. Specifically, the meta-controller considers all nodes in the network as its action space, while the intrinsic controller considers only the neighboring edges of the current node as its action space. Four action selection strategies are designed for path construction, and five reward strategies are designed to guide the agent in constructing the optimal multicast tree.

The points of innovation in this paper are as follows:

- 1) Based on a literature search, we believe that this work presents the first proposal to use the DHRL method to solve the multicast tree problem. An SDN multicast routing mechanism based on DHRL is designed, in which the multicast tree construction process is divided into two subproblems for assignment to the meta-controller and the intrinsic controller of an intelligent agent. The meta-controller solves the problem of selecting fork

nodes, while the intrinsic controller solves the problem of constructing the optimal path from a fork node to a destination node.

- 2) In accordance with the characteristics of the SDN multicast problem, a state space is designed for the meta-controller that consists of a multicast tree state matrix, a link bandwidth matrix, a link delay matrix, and a link packet loss rate matrix. For the intrinsic controller, a state space consisting of the sub-goal matrix representing the position of the fork node and the above four matrices is designed, which enables the intelligent agent to better perceive changes in network status information and changes in the multicast tree construction process.
- 3) In accordance with the characteristics of the two subproblems, corresponding efficient action policies are designed for DHRL at different levels, thereby greatly reducing the selection of invalid actions and making the intelligent agent's search more efficient. The action space of the designed meta-controller is the set of all nodes in the network topology, and each action is the selection of one of them as a fork node. The size of the action space for the designed intrinsic controller is the maximum degree of the network topology, and the output action space tensor of the neural network has a one-to-one correspondence with the neighboring edges of the current node.
- 4) Corresponding reward mechanisms are designed for the different action policies at different levels of DHRL, allowing the intelligent agent to receive more effective feedback and learn more efficiently through interaction with the environment. The reward function of the meta-controller is designed to provide rewards for reaching a single destination node, rewards for reaching all destination nodes, and punishments for invalid fork nodes, driving the meta-controller to select the optimal subgoal in accordance with the current state and pass it down to the lower layer. The intrinsic controller's reward function is designed to include rewards for single-step decisions, rewards for completing path construction, penalties for invalid actions, and penalties for loops, encouraging the intrinsic controller to quickly construct the optimal path from the fork node selected in the upper level to a destination node.

The rest of the paper is organized as follows: Section II reviews related work on multicast routing. Section III presents the problem description and decomposition for SDN multicast routing. Section IV describes the intelligent SDN multicast routing optimization architecture and modeling. Section V details the design of the agent in the proposed DHRL-FNMR algorithm and the implementation for multicast routing. Section VI presents experimental results. Section VII concludes the paper with recommendations for future work.

II. RELATED WORK

This section introduces the relevant methods for solving the optimal multicast tree and Steiner tree problems.

Classical solution methods, including the KMB, MPH, and ADH algorithms, are summarized below. The KMB algorithm [2] constructs a complete graph and utilizes the minimum spanning tree algorithm to obtain a path tree from the source node to the destination nodes in the graph. Subsequently, the shortest path algorithm is used to replace the edges of the path tree in the complete graph to obtain an approximate optimal Steiner tree. The MPH starts with a multicast tree set that includes only the source node. Subsequently, it uses the shortest path algorithm to add the shortest path to the multicast tree set until all destination nodes are included. The ADH algorithm [10] starts from a multicast tree set that contains both the source node and all destination nodes. Subsequently, the algorithm selects the node with the minimum distance to connect any two disconnected trees until all trees are joined. He et al. proposed a heuristic algorithm for constructing reliable multicast trees with multiple sources, considering bandwidth, delay, and reliability constraints to ensure the quality of service (QoS) for multicasting in virtual network functions. They utilized the Floyd algorithm and the Prim algorithm [11] for multicast tree construction. Shi et al. improved the shortest path selection in the KMB algorithm by selecting the path with fewer additional nodes (switches) between the shortest path and the second shortest path, thereby reducing the number of flow entries at SDN switches [12]. Li et al. abstracted the problem as a betweenness-centrality-to-bandwidth ratio tree (BCBT) problem [13]. They used the shortest path tree to generate multicast routes under low traffic and a heuristic greedy algorithm to solve the BCBT optimization problem under link congestion. Bijur et al. used the Dijkstra shortest path algorithm to construct multicast paths based on the receiving devices and network capacity and handled dynamic device joining and leaving to reduce tree changes [14]. Hwang et al. proposed a wireless shortest path heuristic (W-SPH) mechanism for constructing multicast trees in wireless networks [15]. Sun et al. proposed a multicast routing algorithm for QoS assurance in network function virtualization for mobile edge computing, using the shortest path tree algorithm to construct multicast trees [16]. Sufian et al. proposed an energy- and velocity-based multicast protocol for ad hoc networks, in which each destination node selects the route with the highest weight based on route-request packets [17]. These algorithms have high computational complexity and slow convergence speeds, and they have difficulty obtaining the optimal solution as the problem size increases.

Kotachi et al. used integer linear programming to minimize the total number of flow table entries [18]. Latif et al. used integer linear programming to improve the utilization of the fabric in CLOS networks [19]. Touihri et al. employed mixed integer linear programming to improve the minimum residual bandwidth and minimize the number of links in CamCube data center networks based on SDN [20]. Risso et al. proposed a flow-based mixed integer programming formulation for modeling and solving the problem of QoS-aware multicast trees [21]. However, the broad applicability of these methods may be constrained due to the high computational complexity of linear programming and the specific constraints designed for certain scenarios.

Intelligent optimization algorithms usually employ iterative procedures to solve optimization problems. Hamed et al. utilized an ACO algorithm to optimize routing problems. However, the computation time required was high [4]. Zhang et al. employed the leaf crossover genetic algorithm (LCGA), which includes a leaf crossover mechanism that improves the algorithm's performance, but the execution time was long [5]. Shakya and Pulchowk utilized bi-velocity particle swarm optimization (BVDPSO), a dual-speed strategy, to solve multicast routing problems, resulting in faster convergence rates [22]. Sahoo and Kabat combined particle swarm optimization (PSO) and the bacterial foraging algorithm (BFA) to construct QoS-aware multicast routing solutions [23], but the iteration time required by the algorithm was long. To construct multicast trees with delay and bandwidth constraints, Murugeswari et al. proposed a PSO algorithm combined with a GA [24]. Zhang et al. proposed a hybrid ACO and cloud model (CM) algorithm to improve search efficiency and avoid becoming trapped in local optima [25]. Liu et al. employed a noncoding GA to solve application-layer multicast tasks. In this algorithm, genotype is directly represented as a tree-like phenotype and is recombined and mutated to adapt to a tree-like genotype [26]. Despite their effectiveness, these algorithms encounter a common challenge when multiple multicast trees need to be built simultaneously: the computational power required is significant, making it impractical to compute routing strategies for each new flow in an SDN controller [27].

RL approaches enable the construction of optimal paths through the interaction between an agent and the environment. Numerous studies have explored the use of RL methods for unicast routing. Hendriks et al. used a Q-learning algorithm to construct QoS-compliant unicast paths in ad hoc wireless networks. Each node selected a neighbor based on their Q-values for learning to forward a Q-info packet [28]. Casas-Velasco et al. proposed using a Q-learning algorithm to address the unicast routing challenge in SDN. In this algorithm, the state space consists of the source-destination pairs, while neighboring nodes are used as the action space. The reward is determined by factors such as the remaining bandwidth, delay, and packet loss rate [6]. An improvement was later suggested in which a deep Q-network (DQN) agent is used to select one of the k paths in real time for building unicast paths [29]. Guo et al. designed a QoS-aware secure routing protocol using the deep deterministic policy gradient (DDPG) algorithm to construct unicast paths in the SDN-based Internet of Things (IoT) paradigm. The state space contains information such as the current packet arrival frequency, flow table occupancy rate, and occupancy rates between switches and the SDN controller. Meanwhile, the action space includes all switches in the network topology, and an action is chosen based on the probability of each action [30]. Yu et al. proposed a DDPG-based algorithm for developing unicast routes in SDN. In this algorithm, the state space is established as the current traffic matrix, while the action space is the weight of each link. The shortest path algorithm is used to determine the route, and rewards are calculated based on delay, throughput, hop count, and other metrics [31]. Kim et al. and Ning et al.

reassigned various values to links in the network topology and utilized conventional shortest path algorithms to determine routing paths [32], [33]. Nevertheless, the above techniques are specific to unicast situations, and their designated state spaces, action spaces, and reward mechanisms cannot be directly applied to solving multicast routing problems.

The FORMS algorithm was proposed by Forster et al. for the construction of multicast trees in wireless sensor networks (WSNs) using Q-learning. Initially, each node announces its presence by broadcasting, and the nodes then exchange rewards using DATA packets [34]. However, the size of the state space and action space in Q-learning is a drawback. Heo et al. used a DQN to solve the multicast routing problem in SDN but only considered optimizing the length of the multicast tree [35]. Chae and Kim suggested using the Advantage Actor-Critic (A2C) algorithm to handle multicast problems for network topologies that frequently change [36]. However, these authors also solely focused on optimizing the length of the multicast tree without considering other network information. Tran et al. proposed the DQMR protocol [37], which constructs unicast paths from the source node to a destination node in a mobile ad hoc network (MANET) multiple times using a DQN and then combines the constructed paths to form a multicast tree, thereby ensuring the QoS for data transmission. However, this approach requires reconstructing the path from the origin node each time when building the multicast tree.

III. SDN MULTICAST PROBLEM DESCRIPTION

A. Steiner Tree Problem

The aim of this paper is to address the issue of multicast data transmission, which entails transmitting data from one source node to multiple destination nodes. A topology that connects a source node as the root node to multiple destination nodes as the leaf nodes via intermediary nodes is known as a multicast tree or Steiner tree. The minimum Steiner tree, which is the tree that provides the optimal performance among all possible trees, is the primary focus of this paper [38].

Consider a graph $G = (V, E, c)$, where V is the set of nodes in graph G , E is the set of edges, and c is the cost function of each edge. n represents the number of nodes in graph G , $n = |V|$. m represents the number of edges in graph G , $m = |E|$. e_{ij} represents the edge from node i to node j , $e_{ij} \in E$. S denotes the multicast node set, $S \subseteq V$. $S = s \cup D$, where s is the source node and D is the set of destination nodes, $D = \{d_1, d_2, \dots\}$. G' represents a subgraph, that is, a subset of G .

The minimum Steiner tree is the minimum spanning tree T of a subgraph G' that includes paths between s and all nodes in D , where $s \in V'$, $D \in V'$, and the total cost of the tree, $\sum_{e \in T} c(e)$, is minimum. Here, $T = (V', E')$ and $S \subseteq V' \subseteq V$, and $E' \subseteq E$.

Consider a scenario such as a multi-user online interactive videoconference, as shown in Fig. 1, where a user who sends a message corresponding to the source node of a multicast tree may also become a user who is likely to receive the message at the next time corresponding to a destination node. The

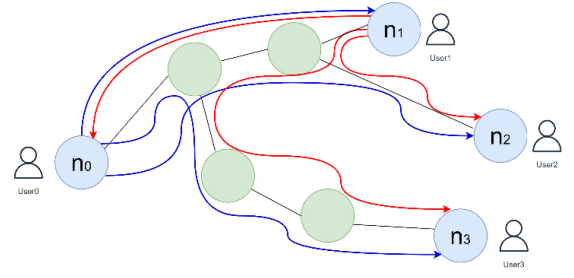


Fig. 1. A Steiner Multicast Tree for multi-user online interactive videoconferencing.

destination node may also become a source node, so that these nodes can alternatively send data in both directions. Therefore, while the source node of a multicast tree sends a message to each destination node, any of the remaining destination nodes may also become a source node to send a message to the remaining nodes. For the sake of clarity in what follows, and integrated with the previously introduced Steiner Multicast Tree definitions, we tentatively call such a Steiner Tree as a Steiner Multicast Tree.

It is assumed that the discussion under such a scenario only serves to facilitate the formulation of the subsequent optimization objectives and solution methods, but it is certain that the subsequent methods developed in this paper are not only applicable to the discussion of such Steiner tree scenarios, but as long as different optimization objectives are set under different scenarios, the proposed methods in this paper are still applicable.

B. Decomposition of the Multicast Problem

Multicast communication involves transmitting data from a single source node to multiple destination nodes via a tree structure with a unique path from the source node to each destination node. In this tree structure, each fork node has a unique path to a destination node, which can be reached either directly from that fork node or by passing through one or more other fork nodes before taking another unique path to the destination node. As a result, a path connecting to each destination node can be established. We employ hierarchical RL to abstract the process of constructing a multicast tree into two subproblems. The first subproblem is to select the next fork node and designate it as the new starting point. The second subproblem requires establishing the optimal one-to-one unicast paths from a fork node to each destination node.

Hierarchical RL (HRL) extends the Markov decision process (MDP) model in RL to an SMDP. According to the hierarchical DQN (h-DQN) algorithm, the value function in RL is also generalized to consider goals and states, and the learning process is abstracted into subgoal learning and control policy learning. The agent employs a two-layer structure consisting of a meta-controller and an intrinsic controller [39].

The meta-controller receives a state s_t as an input and outputs a subgoal $g_t \in \mathcal{G}$, where $\mathcal{G}$ is the set of all available subgoals at time t . The intrinsic controller takes the state s_t and subgoal g_t as inputs to produce an action a_t as an output. As shown by the blue solid lines in Fig. 2, the agent

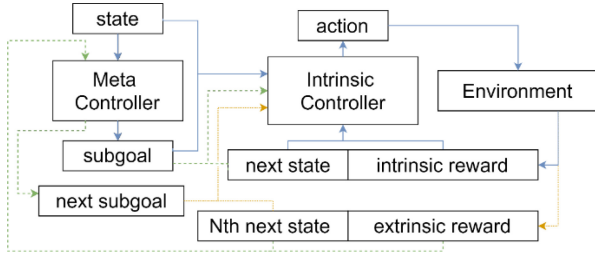


Fig. 2. The structure of the h-DQN algorithm.

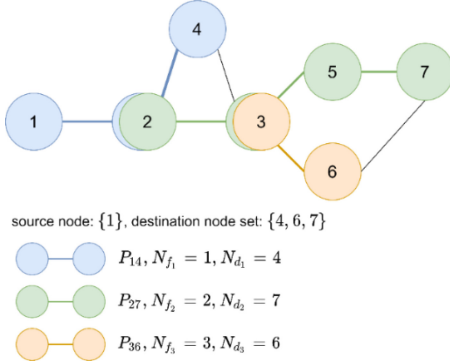


Fig. 3. Constructing a multicast tree using fork nodes.

interacts with the environment by carrying out the intended action a_t , and the environment then provides the agent with an intrinsic reward value r_{t+1}^{in} and transitions to the next state s_{t+1} . Afterward, as shown by the green dashed lines in Fig. 2, the intrinsic controller chooses the next action a_{t+1} based on the next state s_{t+1} and the same subgoal g_t . This iterative process continues until the N -th step, in which the subgoal is accomplished, yielding an extrinsic reward value r_{t+N+1}^{ex} and causing the environment to enter the state s_{t+N+1} . The meta-controller then chooses the next subgoal g_{t+N+1} based on the state s_{t+N+1} , and the iterative process continues in this way until the agent achieves the maximum reward value, as represented by the orange dashed lines in Fig. 2.

The meta-controller is responsible for selecting a fork node N_f from the set of nodes V . In contrast, the intrinsic controller is responsible for selecting the next-hop node as the action starting from N_f . The agent continues to make these selections until it discovers a unicast path P_{fd} connecting N_f to a destination node N_d , where N_d and N_f are elements of V and the destination set D , respectively. By repeatedly doing this, the agent ultimately constructs a multicast tree T , represented by a set of unicast paths, $P_{f_1 d_1}, P_{f_2 d_2}, \dots, P_{f_k d_k}$. Here, f_k represents the k th fork node selected, while d_k represents the k th destination node.

Fig. 3 illustrates the construction of a multicast tree T with the source node $s = 1$ and the destination node set $D = \{4, 6, 7\}$. The first fork node on the path from N_1 to N_4 is the initial node N_1 ($N_{f_1} = 1$), and the destination node is N_4 , making this path P_{14} . The second branching node is N_2 ($N_{f_2} = 2$), and the destination node is N_7 , yielding P_{27} . Finally, the third branching node is N_3 ($N_{f_3} = 3$), and the

destination node is N_6 , resulting in P_{36} . Hence, we construct the multicast tree, $T = \{P_{14}, P_{27}, P_{36}\}$.

Here we consider the Steiner Multicast Tree Scenario mentioned in Section III-A to optimize bw_{tree} , $delay_{tree}$ and $loss_{tree}$. If a multicast tree is denoted as $T = (V_T, E_T)$, where V_T denotes all the nodes in tree T and E_T denotes all the edges of tree T , bottleneck bandwidth is the minimum of all the link bandwidths of a multicast tree, which limits the transfer capacity of the tree. Cumulative delay is the sum of all the delays in the paths from the source node to the destination nodes in a multicast tree. The optimization objective of the meta-controller is to maximize the bottleneck bandwidth (bw_{tree}) and minimize the cumulative delay ($delay_{tree}$) and packet loss rate ($loss_{tree}$) of the tree. Because these objectives may not be mutually independent, they are modeled as a multiobjective optimization problem, as shown in Eq. (1):

$$\max f_1(T), \min f_2(T), \min f_3(T) \quad (1)$$

where $f_1(T)$, $f_2(T)$ and $f_3(T)$ are shown in Eq. (2), Eq. (3), and Eq. (4), respectively:

$$f_1(T) = bw_{tree} = \min_{e_{ij} \in T} (bw_{ij}) = \min_{d \in D} (bw_{sd}) \quad (2)$$

$$f_2(T) = delay_{tree} = \sum_{e_{ij} \in T} delay_{ij} \quad (3)$$

$$f_3(T) = loss_{tree} = 1 - \prod_{e_{ij} \in T} (1 - loss_{ij}) \quad (4)$$

where T is a Steiner tree in graph G , bw_{ij} represents the link bandwidth of e_{ij} , and P_{sd} represents the path in multicast tree T from source node s to destination node d , $\min_{d \in D} (bw_{sd})$ represents the Bottleneck bandwidth of path P_{sd} . $delay_{ij}$ denotes the link delay of e_{ij} . $loss_{ij}$ denotes the link packet loss rate of e_{ij} . Eq. (2) represents the maximization of the bottleneck bandwidth in the Steiner multicast tree mentioned in Section III-A, Eq. (3) represents the difference in the delay experienced by the user represented by the end node sending a message in both directions makes the delay as small as possible, and Eq. (4) represents the packet loss rate experienced by the users. This loss rate is represented by the end node sending a message in both directions, making the loss as small as possible.

The optimization objective for the path P_{fd} constructed by the intrinsic controller from the fork node N_f selected by the meta-controller to destination node N_d is given in Eq. (5):

$$\max f_1(P_{fd}), \min f_2(P_{fd}), \min f_3(P_{fd}) \quad (5)$$

where $f_1(P_{fd})$, $f_2(P_{fd})$ and $f_3(P_{fd})$ are shown in Eq. (6), Eq. (7) and Eq. (8), respectively:

$$f_1(P_{fd}) = bw_{P_{fd}} = \min_{e_{ij} \in P_{fd}} (bw_{ij}) \quad (6)$$

$$f_2(P_{fd}) = delay_{P_{fd}} = \sum_{e_{ij} \in P_{fd}} delay_{ij} \quad (7)$$

$$f_3(P_{fd}) = loss_{P_{fd}} = 1 - \prod_{e_{ij} \in P_{fd}} (1 - loss_{ij}) \quad (8)$$

To solve this problem, this paper introduces an algorithm, DHRL-FNMR, that uses DHRL. This algorithm finds paths through fork nodes for constructing multicast trees.

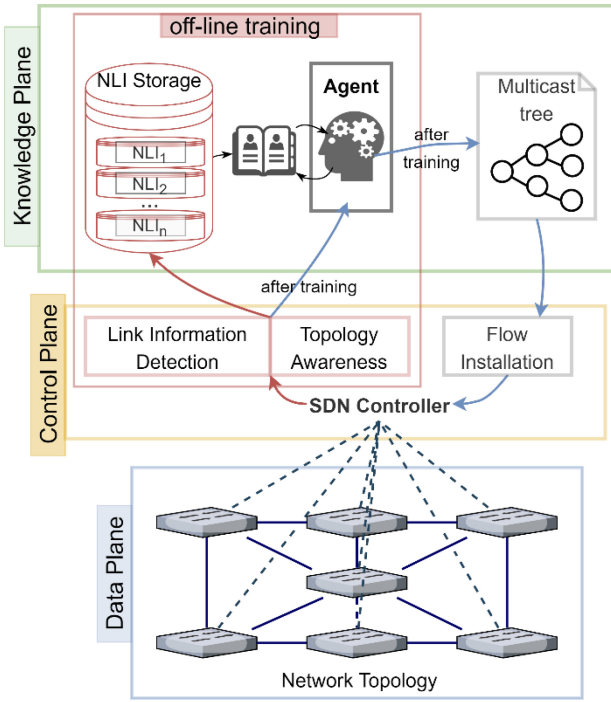


Fig. 4. Intelligent SDN multicast routing architecture.

IV. INTELLIGENT SDN MULTICAST ROUTING OPTIMIZATION ARCHITECTURE

This paper outlines an intelligent multicast routing framework for the DHRL-FNMR algorithm, as depicted in Fig. 4. The framework is composed of three planes, namely, the data plane, control plane, and knowledge plane, similar to the DRL-M4MR framework. In the control plane, an SDN controller interacts with the data plane using the OpenFlow protocol to gain network topology awareness, measure link information, and distribute flow tables. The gathered network link status information (NLI) is stored in the knowledge plane's NLI storage to support the DHRL-FNMR agent's training phase. Upon the completion of agent training, the SDN controller utilizes the current NLI to generate a multicast tree and distribute multicast routing flow tables accordingly. Further design details are explained below.

A. Data Plane

The data plane's OpenFlow switches use the OpenFlow protocol to connect and transmit packets to the SDN controller for routing and forwarding. Within each switch, incoming packets are evaluated against the internal flow table and then forwarded to the appropriate output ports. If the switch matches a multicast address, the packet will be forwarded to one or more ports based on the internal multicast routing flow table; otherwise, the packet will be discarded. Additionally, through the southbound interface, the data plane can respond to requests from the control plane.

B. Control Plane

The control plane encompasses a centralized SDN controller that connects to each switch in the data plane through the

southbound interface, facilitating the collection of global switch and link state information, such as the remaining bandwidth, delay, and packet loss rate of each link. The SDN controller integrates three crucial functions: network topology awareness, link information detection, and flow table distribution.

For network topology awareness, the SDN controller monitors switch state change events in the data plane, based on which it generates the network topology. Additionally, the SDN controller parses Address Resolution Protocol (ARP) packets to obtain host information, including Internet Protocol (IP) addresses and media access control (MAC) addresses, and records each host's location.

The SDN controller measures the remaining bandwidth bw_{ij} , delay $delay_{ij}$, and packet loss rate $loss_{ij}$ of each link in the established network topology. To measure the remaining bandwidth and packet loss rate of a link, the SDN controller initiates a port information query request to the data plane, and after parsing the reply message, it obtains statistics on the number of bytes sent (t_b), the number of bytes received (r_b), the number of packets sent (t_p), the number of packets received (r_p), and the effective time (t_{dur}) for each port. The remaining bandwidth bw_{ij} is deduced via Eq. (9), where bw_c denotes the bandwidth capacity of the link. Eq. (10) determines the packet loss rate $loss_{ij}$ of the link.

$$bw_{ij} = bw_c - \frac{|(t_{b2} + r_{b2}) - (t_{b1} + r_{b1})|}{t_{dur2} - t_{dur1}} \quad (9)$$

$$loss_{ij} = \max\left(\frac{tp_i - rp_j}{tp_i}, \frac{tp_j - rp_i}{tp_j}\right) \quad (10)$$

where, t_{b*} , r_{b*} , and t_{dur*} represent the number of bytes sent, the number of bytes received and the effective time, respectively, at time $*$. tp_* and rp_* denote the numbers of sent and received packets, respectively, at port $*$. The function $\max(\cdot)$ has a physical interpretation regarding packet loss rates. It identifies the maximum value between the loss rate from port i to j and from j to i . This maximum value represents the total packet loss rate between port i and port j .

Liao and Leung [40] proposed Eq. (11) as an approximation for the $delay_{ij}$ of link e_{ij} , which is obtained by measuring the delays of the two links that link each switch to the SDN controller, using ping requests and reply messages (d_{echo1} and d_{echo2}), and the link delay between the switches, derived from LLDP packets (d_{lldp1} and d_{lldp2}).

$$delay_{ij} = \frac{(d_{lldp1} + d_{lldp2} - d_{echo1} - d_{echo2})}{2} \quad (11)$$

The NLI for the entire network topology is collected once every interval t and stored for future use.

One of the primary functions of the control plane is the deployment of flow tables. Once the SDN controller has calculated either a unicast or multicast path, it can accordingly deploy the corresponding flow entries to the data-plane routers, thus enabling data packets to be forwarded from input to output ports. If the destination node's address is a general IP address, the SDN controller deploys unicast flow tables. Otherwise, if the address is a multicast address, multicast path flow entries are deployed. Additionally, the switches located

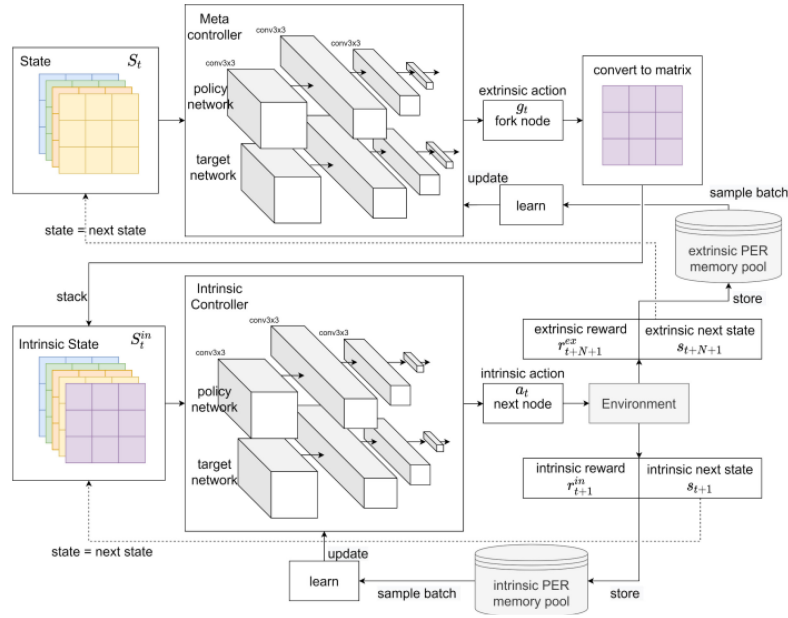


Fig. 5. DHRL-FNMR algorithm framework.

at the fork points in the multicast tree have flow entries set to forward data packets to different designated ports during the deployment of flow tables.

C. Knowledge Plane

This paper principally focuses on the knowledge plane, where the intelligent agent is subjected to offline training using the collected NLI to learn how to construct optimal multicast paths, which will then be transmitted to the control plane. Additionally, the knowledge plane processes the NLI stored in its NLI storage, converting it into matrix forms for input to the meta-controller and intrinsic controller. The state information for the intrinsic controller and meta-controller can be modified in this way, and diverse reward values can be provided as feedback in the programming environment. The model implementation and experience pools are also stored in the knowledge plane. The detailed algorithm design is presented in Section V.

V. DESIGN OF THE DHRL-FNMR INTELLIGENT SDN MULTICAST ROUTING ALGORITHM BASED ON HRL

The algorithm framework designed in this article is shown in Fig. 5. First, based on the current state s_t , i.e., the current multicast tree state matrix, remaining bandwidth matrix, link delay matrix, and packet loss rate matrix, the upper-level meta-controller selects a fork node as the subgoal g_t from all nodes based on the Q-value function. The Q-value function is fitted using a convolutional neural network, and dual networks, i.e., a policy network and a target network, are used to reduce the impact of overestimation.

Then, the selected fork node is transformed into a subgoal matrix and stacked with the state s_t to form the state s_t^{in} provided as input to the intrinsic controller. Based on s_t^{in} and g_t , the intrinsic controller selects the next-hop node as the action a_t . The agent interacts with the environment, which

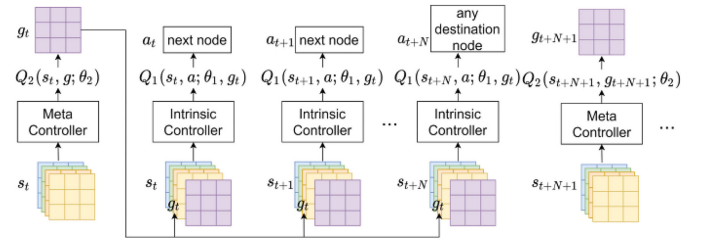


Fig. 6. Learning process of the intrinsic controller and meta-controller.

returns the reward value r_{t+1}^{in} calculated based on the reward function and the next state s_{t+1} to the intrinsic controller depending on the selected action.

After storing the transition $(s_t, g_t, r_{t+1}^{in}, s_{t+1})$ in its internal prioritized experience replay pool, the agent collects a mini-batch from this replay pool for loss calculation, uses gradient descent to update the parameters θ_1 of the internal policy network, and then proceeds to the next state iteration of the intrinsic controller. When the agent reaches any destination node in step N , it is considered to have completed a subgoal, and the environment provides feedback on the next state s_{t+N+1} and the extrinsic reward value r_{t+N+1}^{ex} calculated through the reward function.

The meta-controller's policy network is updated through batch sampling from the external prioritized experience replay pool. This process is similar to the intrinsic controller's learning process. The external prioritized experience replay pool stores transition tuples of the form $(s_t, g_t, a_t, r_{t+N+1}^{ex}, s_{t+N+1})$.

After the meta-controller's policy network is updated, the meta-controller selects a new fork node, g_{t+1} , and the learning process continues iteratively until all destination nodes are reached, completing the agent's learning process. The learning process of both the intrinsic controller and the meta-controller is shown in Fig. 6.

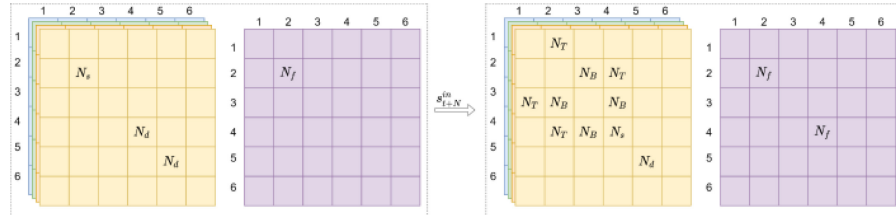


Fig. 7. Design of the state space, where matrices of different colors represent M_{BW} , M_{Delay} , M_{loss} , and M_g .

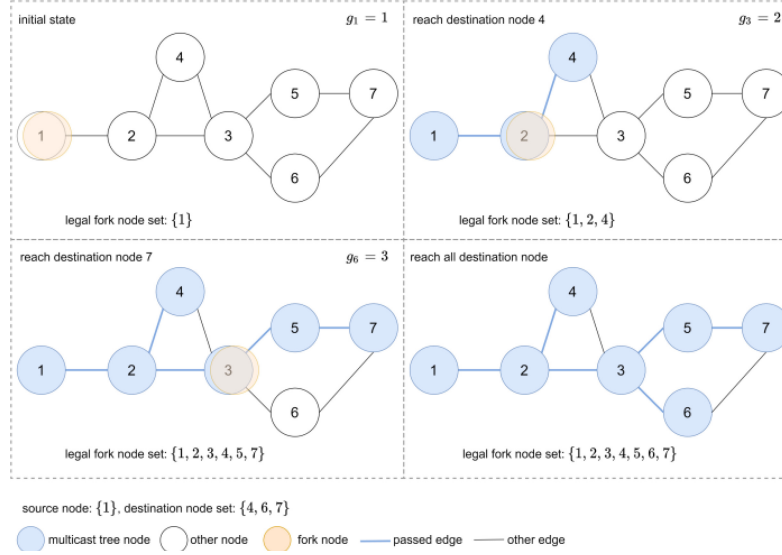


Fig. 8. Design of the meta-controller's action space.

Finally, at fixed intervals, the agent updates the parameters of the target network by setting them equal to those of the policy network. Through training, the agent converges to the maximum reward value, which enables the construction of the optimal multicast tree.

A. State Space

The state space of the meta-controller consists of the link remaining bandwidth matrix M_{BW} , the link delay matrix M_{Delay} , and the link loss rate matrix M_{loss} , which are designed in the same way as in DRL-M4MR, using normalized link information as the matrix elements. In this paper, an additional channel is added, named the subgoal matrix, and the design of the multicast tree state matrix M_T is modified. In M_T , whenever the agent's state moves to the next-hop node, the diagonal element of the corresponding node is marked as N_s , and the mark from the previous step is deleted. When a new fork node is selected, it is also marked as N_s , as shown in the yellow matrix in Fig. 7.

The state space of the intrinsic controller is expanded by introducing the subgoal matrix M_g . Whenever the meta-controller selects a node as a fork node, the corresponding diagonal element in M_g is marked as N_f . As shown in the purple matrix, in the example presented in Fig. 7, N_2 is first selected as the fork node, and then N_3 is selected. Whenever the same link or node is selected repeatedly, the marks are stacked to represent different states of the agent. The intrinsic

controller uses a dimension stacking operation to stack the target matrix onto the channel dimension of the state space of the meta-controller, thus forming the input for the state-input neural network of the intrinsic controller.

B. Action Space

The two controllers in HRL have different learning tasks and are designed with different action spaces: the external action space (the action space of the meta-controller) and the internal action space (the action space of the intrinsic controller). Because the output of a neural network is a tensor with fixed dimensions, each element of this tensor corresponds to a physical meaning. The external action space is the set of all nodes in the network, denoted by $\mathcal{A}_{ex} = V = \{N_1, N_2, \dots, N_n\}$, where n is the number of nodes in graph G . Therefore, each element of the associated tensor corresponds to a node in the graph. When making a decision, the meta-controller selects one of these n nodes as its action, which also serves as the subgoal for the intrinsic controller. This means that the agent selects this node as the fork node and passes the goal of finding a destination node starting from this node to the lower-level intrinsic controller. Since the output encompasses all nodes in the graph, there are two possible scenarios when selecting a fork node:

(1) If the selected fork node is not in the current multicast tree, the state remains unchanged, and the agent is penalized before selecting a new action.

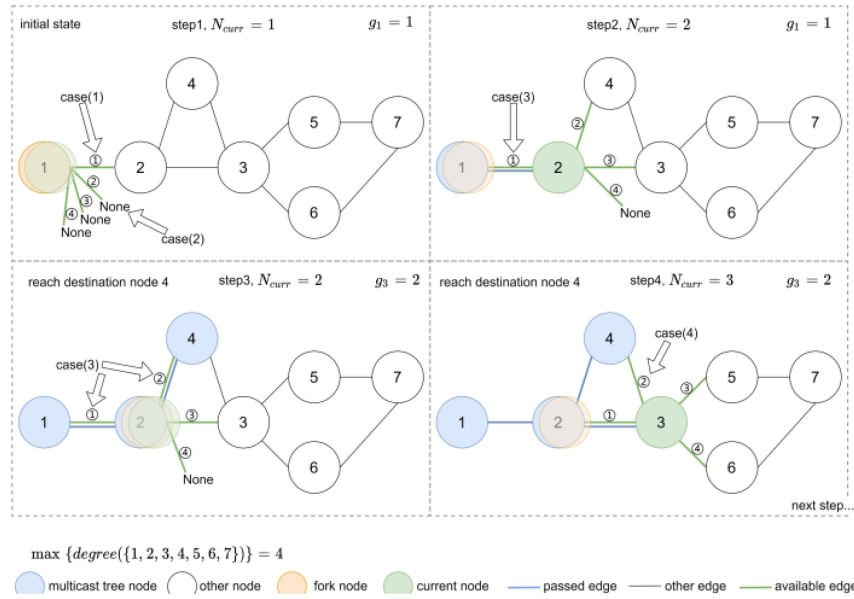


Fig. 9. Design of the intrinsic controller's action space.

(2) If the selected fork node is a node in the current multicast tree, the intrinsic controller can continue to run.

Once any of the destination nodes is reached, a new node is selected as the fork node until all destination nodes are found. Once all destination nodes have been found, the meta-controller terminates the task, which also indicates the completion of multicast tree construction.

Fig. 8 illustrates an example of the selection of actions by the meta-controller. The source node is N_1 , and the destination node set consists of N_4 , N_6 , and N_7 . The blue circles represent the nodes in the multicast tree that has been constructed so far, the orange circles represent the nodes that have been selected as fork nodes, the white circles represent other nodes in the graph, the blue lines represent the links in the multicast tree that has been constructed so far, and the black lines represent other links in the graph. In the initial state, only the source node can be selected as the fork node. Therefore, N_1 is selected as the fork node, i.e., $g_1 = 1$, and then the intrinsic controller searches for a destination node. Suppose that the intrinsic controller reaches the destination node N_4 after making two decisions. The meta-controller then chooses a new fork node. If $g \in \{1, 2, 4\}$, this fork node is considered valid, and the intrinsic controller continues to learn; otherwise, it is punished. Suppose that N_2 is chosen as the fork node; then, $g_3 = 2$. The intrinsic controller continues to make decisions, and when it reaches the destination node N_7 after three steps, the meta-controller can then choose from the subgoal set $\{1, 2, 3, 4, 5, 7\}$. At this time, if $g \in \{1, 2, 3, 4, 5, 7\}$, it is considered valid. Suppose that N_3 is now chosen as the fork node; then, $g_6 = 3$. The intrinsic controller starts from N_3 and finds the destination node N_6 . The agent can no longer choose a next-round action because all destination nodes have been reached.

The internal action space is denoted by $\mathcal{A}_{in} = EN_i = \{e_{ij_1}, e_{ij_2}, \dots, e_{ij_l}\}$, where EN_i represents the set of edges adjacent to the current node $N_i \in V$ and l denotes the

maximum degree of graph G . Each element of the associated tensor corresponds to an adjacent edge of the current node, and a tensor element is considered Null when there is no corresponding edge. After each action selection by the agent and interaction with the environment, there will be four possible scenarios:

- (1) If the agent has selected an edge that can be used for the next hop, the current state will transition to the next state S_{t+1} , and a positive reward will be obtained.
- (2) If the agent has selected an edge that has no corresponding edge, the current state will not change, and an invalid selection penalty will be obtained.
- (3) If the agent has selected an edge that has already been added to the multicast tree, the corresponding flag will be added, and the agent will enter the next state while receiving a loop penalty.
- (4) If the selected edge forms a cycle, the agent will enter the next state and receive a cycle penalty.

Fig. 9 shows a graph with N_1 as the source node and N_4 , N_6 , and N_7 as the set of destination nodes. The nodes and links are color-coded: blue circles and lines represent nodes and links already in the multicast tree, orange circles represent fork nodes, green circles and lines represent the agent's current location and available links, and white circles and black lines represent other nodes and links in the graph. Since the maximum degree in the graph is 4, the neural network outputs a tensor with dimensions of (1×4) , with each element corresponding to one of the (up to) four edges adjacent to the current node.

In the initial state, the agent is at node N_1 , with only one available edge, e_{12} , and the other three elements in the tensor are Null. If the agent selects the first element of the tensor, i.e., e_{12} , as the action, this corresponds to situation (1) specified above, and the current node of the agent will change to N_2 ; if any other edge is selected, this corresponds to situation (2).

In the second step, the current node is N_2 . The first three elements of the output tensor represent edges e_{21} , e_{24} , and e_{23} , and the fourth element is Null. Since edge e_{21} is already in the multicast tree, selecting e_{21} as the action corresponds to situation (3) above. As the agent makes decisions, the current node changes with each decision.

In the fourth step, the current node is N_3 . At this time, the four dimensions of the output tensor represent edges e_{32} , e_{32} , e_{35} , and e_{36} , and the four elements correspond to these four edges. If the agent selects e_{34} as the action, this corresponds to situation (4) described above.

C. Reward Function

The reward function determines the learning direction of the agent, and our objective is for the meta-controller to learn the optimal fork node selection strategy, while the intrinsic controller is expected to learn the optimal strategy for finding a path from a fork node to a specific destination node.

Two types of extrinsic reward functions are defined, one for completing subgoals and one for selecting illegal fork nodes:

(1) If the agent reaches a certain destination node from a fork node, it will receive a reward value R_g , as shown in Eq. (12).

$$R_g = \beta_1 bw_{P_{ij}} + \beta_2 (1 - delay_{P_{ij}}) + \beta_3 (1 - loss_{P_{ij}}) \quad (12)$$

Here, P_{ij} represents the unicast path from fork node N_i to destination node d_j . $bw_{P_{ij}}$ represents the bottleneck bandwidth of the path, which is calculated in the same way as bw_{tree} . $delay_{P_{ij}}$ represents the delay of the path, which is calculated in the same way as $delay_{tree}$. $loss_{P_{ij}}$ represents the path's packet loss rate, which is calculated in the same way as $loss_{tree}$.

If the agent successfully reaches all destination nodes, it will receive R_{finish} , calculated as shown in Eq. (13).

$$R_{finish} = \beta_1 bw_{tree} + \beta_2 (1 - delay_{tree}) + \beta_3 (1 - loss_{tree}) \quad (13)$$

(2) When the agent selects an illegal fork node, it will be given a fixed negative penalty value, $R_{illegal} = C_1$.

If the subgoal is not completed, that is, neither of the above two situations is satisfied, then no reward value is provided, and the intrinsic controller learns at this time. The intrinsic reward value is generated based on the interaction between the agent and the environment. We define four types of intrinsic reward functions based on the state of the multicast tree that may be formed after an action is executed.

(1) If the agent selects the correct next-hop node and the next-hop node is not a destination node, then it will receive a reward value R_{step} , which is calculated in the same way as in DRL-M4MR, as shown in Eq. (14).

$$R_{step} = \beta_1 bw_{ij} + \beta_2 (1 - delay_{ij}) + \beta_3 (1 - loss_{ij}) \quad (14)$$

(2) If there is no corresponding link for the selected action, then a constant penalty value $R_{none} = C_2$ is given.

(3) If a valid link is selected as the next-hop link but the selected link already exists in the multicast tree or selecting it

would create a loop, then a constant penalty value $R_{back} = C_3$ is given.

(4) If the agent reaches any destination node after selecting a link, it is considered to have completed the current subgoal, and the final reward value R_g is calculated and returned based on the path P_{fd} taken.

In the above reward functions, to prevent any single factor from having too much impact on the reward value, we use the same normalization method as in DRL-M4MR.

D. Q-Value Function

The HRL algorithm used in this paper is h-DQN, which is an improved algorithm based on the DQN and max-Q algorithms. The intrinsic controller of h-DQN additionally includes a parameter target g compared to the traditional DQN algorithm and evaluates the Q-value in accordance with Eq. (15):

$$\begin{aligned} Q_1^*(s, a; g) &= \max_{\pi_{ag}} E \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1}^{in} | s_t = s, a_t = a, g_t = g, \pi_{ag} \right] \\ &= \max_{\pi_{ag}} E \left[r_{t+1}^{in} + \gamma \max_{\pi_{ag}} Q_1^*(s_{t+1}, a_{t+1}; g) | \right. \\ &\quad \left. s_t = s, a_t = a, g_t = g, \pi_{ag} \right] \end{aligned} \quad (15)$$

where g represents the target of the agent in state s and π_{ag} represents the executed action policy. This equation evaluates the value of executing the selected action in the current state under the current policy. The Q-value function of the meta-controller is similar to that in the DQN algorithm, as shown in Eq. (16):

$$\begin{aligned} Q_2^*(s, g) &= \max_{\pi_g} E \left[\sum_{k=0}^N r_{t+k+1}^{ex} + \gamma \max_{g'} Q_2^*(s_{t+N}, g') | s_t = s, g_t = g, \pi_g \right] \end{aligned} \quad (16)$$

where N represents the number of time steps taken by the controller to complete the given target, g' represents the target of the agent in state s_{t+N} , and π_g represents the policy under the current target.

Accordingly, the transition obtained by the meta-controller based on Q_2 is represented as $(s_t, g_t, r_{t+N}^{ex}, s_{t+N})$, which is stored in the experience pool PER_2 . On the other hand, the transition obtained by the intrinsic controller based on Q_1 is represented as $(s_t, a_t, g_t, r_{t+1}^{in}, s_{t+1})$, which is stored in PER_1 .

The loss function used to update the parameters of Q_1 and Q_2 is the Huber loss function, as shown in Eq. (17):

$$L_n = \begin{cases} 0.5(x_n - y_n)^2, & \text{if } |x_n - y_n| < 1 \\ |x_n - y_n| - 0.5, & \text{otherwise} \end{cases} \quad (17)$$

$$y_n = r + \gamma \max_{a'} Q(s', a'; \theta, g) \quad (18)$$

where s' and a' denote the next state and next action, respectively. For Q_1 , x_n is the predicted Q_1 value and y_n is the expected Q_1 value for the n th sample in the minibatch, and the same applies for Q_2 .

E. Exploration Method

The decaying ε -greedy method is used as the exploration strategy. In the early stages of training, the agent attempts to sample diverse actions in the action space to explore and learn from more experiences, as shown in Eq. (19):

$$a_t = \begin{cases} \operatorname{argmax}_a Q_t(s, a), & \text{if } x \geq \varepsilon \\ \text{random choice}, & \text{otherwise} \end{cases} \quad (19)$$

where x follows a uniform distribution between 0 and 1, denoted by $x \sim U(0, 1)$. The value of ε is calculated in accordance with Eq. (20):

$$\varepsilon = \varepsilon_{\text{final}} + (\varepsilon_{\text{start}} - \varepsilon_{\text{final}}) \cdot \exp\left(-\frac{\text{epoch}_{\text{curr}}}{\text{epoch}_{\text{decay}}}\right) \quad (20)$$

Here, $\varepsilon_{\text{start}}$ represents the initial value of ε at the beginning of training. $\text{epoch}_{\text{curr}}$ represents the current epoch or training iteration, while $\text{epoch}_{\text{decay}}$ controls the convergence epoch, with smaller values leading to earlier convergence. As training progresses, ε approaches $\varepsilon_{\text{final}}$. Both $\varepsilon_{\text{start}}$ and $\varepsilon_{\text{final}}$ are in the range [0, 1].

F. Prioritized Experience Replay

For the experience replay buffers, the same prioritized experience replay design is used as in DRL-M4MR. In contrast to uniform sampling from a typical experience buffer, in a prioritized experience replay (PER) buffer, transitions with a larger temporal difference error (TD-error) have a higher probability of being sampled. The agent learns more from these transitions, as they are more likely to contain valuable learning experiences [41]. PER allows the agent to more easily sample useful learning samples instead of obtaining many negative samples associated with penalty values, making the learning process more effective.

The probability of sampling is proportional to the TD-error, i.e., $i \sim P(i) \propto |TD_{\text{error}}|^\alpha$, as shown in Eq. (21), where α is a hyperparameter that determines the shape of the distribution ($\alpha = 0$ corresponds to uniform sampling) and p_i is a priority index determined by the TD-error. Finally, an importance sampling parameter, ω_i , is used to correct the distribution bias, as shown in Eq. (22), where N is the size of the experience pool and β is the importance sampling hyperparameter.

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \quad (21)$$

$$\omega_i = (N \cdot P(i))^{-\beta} \quad (22)$$

G. h-DQN-Based Multicast Routing Algorithm

The agent of DHRL-FNMR is trained based on the collected network environment information, using the source node N_s and the destination node set N_D as input to find the maximum reward value as the target, in order to construct the optimal multicast tree. As shown in Algorithm 1, the input provided to the agent includes the source node, N_s ; the destination node set, N_D ; the total number of training epochs, M ; and hyperparameters of the intrinsic controller and meta-controller, such as the learning rate α , the batch size k for sampling, the target network parameter update frequency n_{update} , the

number of epochs until convergence for the decaying ε -greedy method, and the hyperparameters of the internal and external PER pools. The output is a multicast tree T in graph G from N_s to N_D .

Lines 1 to 3 correspond to model parameter initialization and PER pool initialization. The parameters of the policy networks for the intrinsic controller and meta-controller, denoted by θ_1 and θ_2 , respectively, are initialized first. Then, the neural network parameters of the target networks, $\hat{\theta}_1$ and $\hat{\theta}_2$, are initialized to the parameters of the corresponding policy networks. The internal and external PER pools are then initialized based on the experience pool size, M_m .

After initialization, the agent begins training for a total of M iterations, as shown in lines 4–39. Lines 5–7 show that the remaining bandwidth, delay, and loss rate of each network link are obtained from the NLI repository for each time step and are then converted into matrix form for the state space, corresponding to M_{BW} , M_{delay} , and M_{loss} , respectively. The multicast tree status matrix M_T for the initialized environment is then stacked in the channel dimension to form a matrix with dimensions of $(1 \times 4 \times n \times n)$, which serves as the representation of the current state s_t for the meta-controller.

Lines 8–37 represent the outer meta-controller loop. First, on line 9, the meta-controller obtains the subgoal g_t based on the decaying ε -greedy method for the current state s_t . On line 10, the environment determines whether g_t is a valid subgoal, and if so, it is converted into matrix form (M_{g_t}) and enters the inner controller training loop (lines 11–30).

In lines 12–14, first, the state s_t considered by the meta-controller and the subgoal matrix M_{g_t} are stacked along the channel dimension to form the input to the inner controller. The action a_t is obtained based on the decaying ε -greedy method. Then, on line 14, the agent interacts with the environment by executing the action a_t . The environment provides feedback in the form of the next state s_{t+1} , the reward value r_{t+1}^{in} generated through the intrinsic controller's interaction with the environment, the reward value r_{t+1}^{ex} generated through the meta-controller's interaction with the environment, a state flag indicating the state after the intrinsic controller executes the action, and a subgoal flag indicating whether the subgoal has been achieved. On line 15, the transition generated by the intrinsic controller is stored in the internal PER pool PER_2 .

The intrinsic controller then learns from its previous experience on lines 16–22. First, a batch of size k is sampled from PER_2 and stacked to form a $(k \times 4 \times n \times n)$ input to the policy network to obtain the Q_1 value. The expected Q -value is calculated, followed by the TD-error. The priority of the transition in the PER pool is updated based on the TD-error. Then, the loss value is computed using the Huber loss function, and the internal network parameters are updated using gradient descent. The pseudocode for the learn() function is the same for both the intrinsic controller and the meta-controller.

On lines 23–27, if the subgoal has been achieved, the transition is stored in PER_1 , and the meta-controller learns from its experience. Finally, the intrinsic controller loop is exited, and the agent searches for a new subgoal to train on. Otherwise, on line 28, the agent moves to the next state and

Algorithm 1: HDRL-FNMR

Input: source node N_s , destination nodes set. N_D learning rate α , reward decay γ , batch size k , update frequency n_{update} , training episodes M .

Output: optimal multicast tree for (N_s, N_D) .

- 1 Initialize intrinsic controller's and meta controller's policy network with random weight θ_1 and θ_1 ;
- 2 Initialize intrinsic controller's and meta controller's target network with weight $\hat{\theta}_1 = \theta_1, \hat{\theta}_2 = \theta_2$;
- 3 Initialize PER pool PER_1 and PER_2 ;
- 4 **for** $episode = 1 \leftarrow M$ **do**
- 5 **for** $M_{BW}, M_{delays}, M_{loss}$ in $NLIs$ Storage **do**
- 6 Reset environment with (N_s, N_D) ;
- 7 Get $s_t = stack(M_T.M_{BW}, M_{delay}, M_{loss})$;
- 8 **while** *True* **do**
- 9 // meta controller loop.
- 10 $g_t = meta_controller.sample(s_t)$, according to decay ϵ -greedy;
- 11 **if** $env.subgoal_step(g_t)$ **then**
- 12 // g_t is legal.
- 13 **while** *True* **do**
- 14 // intrinsic controller loop.
- 15 $s_t^{in} = stack(s_t, M_{gt})$;
- 16 $a_t = intrinsic_controller.sample(s_t^{in})$, according to decay ϵ -greedy;
- 17 // Agent execute action a_t , observe reward $r_{t+1}^{in}, r_{t+1}^{ex}$, and next state s_{t+1} .
- 18 $s_{t+1}, r_{t+1}^{in}, r_{t+1}^{ex}, state_flag, subgoal_flag = env.step(a_t)$;
- 19 $intrinsic_controller.store(s_t^{in}, a_t, r_{t+1}^{in}, s_{t+1})$, store transition in PER_1 ;
- 20 // $intrinsic_controller.learn()$ as follows.
- 21 Sample batch $(s_i, a_i, r_{t+1}^{in}, s_{t+1})$ and get ω_i from $PER_1, i = 1, 2, \dots, k$;
- 22 // ω_i is importance-sampling weight.
- 23 Get $Q_i(s_i, a_i)$ from intrinsic policy network;
- 24 **if** s_{t+1} is not *None* **then** $R' = r_{t+1}^{in} + \gamma max_{a'} \hat{Q}_1(s_{t+1} + a')$;
- 25 **else** $R' = r_{t+1}^{in}$;
- 26 Compute TD-error $\delta_i = R' - Q_1(s_i, a_i)$;
- 27 Update PER_1 transition priority by δ_i ;
- 28 Update θ_1 by performing gradient descent on $Huber(\theta_1)$ and ω_i ;
- 29 **if** $subgoal_flag$ is *True* **then**
- 30 // reach any destination.
- 31 $meta_controller.store(s_t, g_t, r_{t+1}^{ex}, s_{t+1})$, store transition in PER_2 ;
- 32 $meta_controller.learn()$, calculate loss and update θ_2 ;
- 33 **break**;
- 34 **end**
- 35 $s_t = s_{t+1}$;
- 36 In every n_{update} steps update target network, set $\hat{\theta}_1 = \theta_1, \hat{\theta}_2 = \theta_2$;
- 37 **end**
- 38 // The agent reaches all destinations.
- 39 **if** $state_flag$ is *True* **then** **break**;
- 40 **end**
- 41 **else**
- 42 $meta_controller.store(s_t, g_t, R_{illegal}, s_{t+1})$, store transition in PER_2 ;
- 43 $meta_controller.learn()$, calculate loss and update θ_2 ;
- 44 **end**
- 45 **end**
- 46 **end**
- 47 Use final policy networks with parameter θ_1 and θ_2 to construct multicast tree from N_s to N_D the agent execute the max Q-value action for each decision.

continues training. On line 29, the parameters of the target networks are updated every n_{update} time steps by setting them equal to those of the corresponding policy networks. Line 31 is reached when the agent has reached all destination nodes; at this time, the meta-controller loop is exited to start the next training session.

If the subgoal g_t is not valid, lines 34 and 35 are executed. The meta-controller receives the reward value $R_{illegal}$, and the

next state is set to Null. The transition is stored in PER_1 , and the network parameters are updated by computing the loss value.

Finally, after the agent completes its training, it selects the optimal Q -value from the policy networks of the intrinsic controller and meta-controller corresponding to the current state to construct the optimal multicast tree from N_s to N_D .

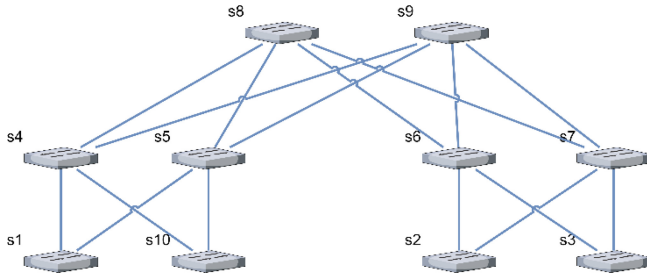


Fig. 10. Fat-Tree Network Scenario with 10-switches and 16-links.

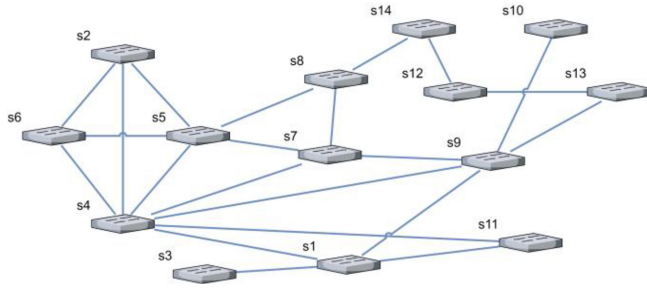


Fig. 11. Node14Net Scenario with 14-switches and 23-links.

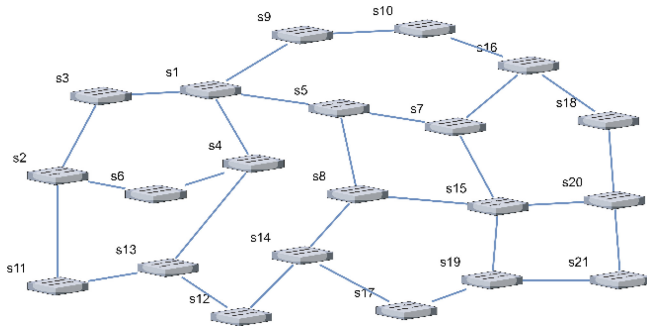


Fig. 12. ARPA-Net Scenario with 21-switches and 29-links.

VI. EXPERIMENTS AND EVALUATION

The experimental control server used in this study was an Ubuntu 20.04.3 server equipped with a GeForce RTX 3090 graphics card. The SDN controller employed was Ryu, and the data plane was simulated using Mininet 2.3.0. Ryu provided the southbound interface for communication with Mininet's Open vSwitches via OpenFlow 1.3. The implementation of HRL and deep neural networks was carried out using Python 3.8 and PyTorch 1.9.0.

In order to evaluate the performance of the multicast algorithm proposed in this paper as comprehensively as possible, comparative experiments were carried out in the following three topological structures: Fat-Tree Network, Node14Net and ARPA-Net as test scenarios. The details are as follows: Fat-Tree Network, as shown in Fig. 10, is a topology reported in the literature [42], and is a fat-tree structure composed of 10 switches and 16 links. Node14Net, as shown in Fig. 11, is a topology that comes from the website [43], and is a data center network topology in New York City, including 14 switches and 23 links. ARPA-Net, as shown in Fig. 12, is a topology reported in the literature [42], and is the Advanced Research Projects Agency Network, including 21 switches and 29 links.

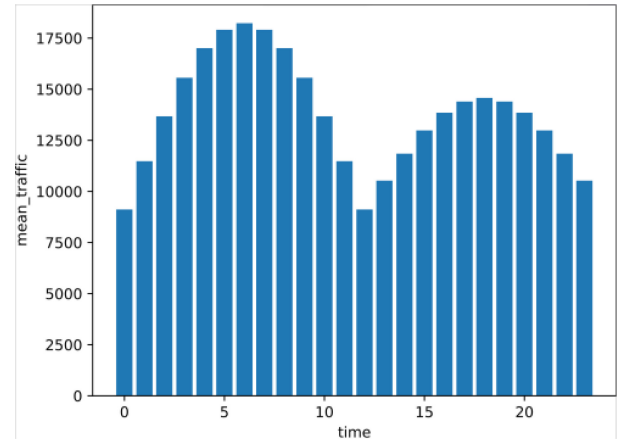


Fig. 13. Distribution of generated traffic flow.

In the Fat-Tree Network scenario, the source node of the multicast tree is node 3 and the destination nodes are nodes 7, 8 and 10. In the Node14Net scenario, the source node of the multicast tree is node 12 and the destination nodes are nodes 2, 4 and 11. In the Node14Net scenario, the source node of the multicast tree is node 3 and the destination nodes are nodes 7, 8 and 13.

For the above three topological scenarios, we use the traffic generation tool *Iperf3* to simulate the traffic situation for 24 hours in a day, as shown in Fig. 13. The abscissa represents time, and the ordinate represents the average traffic sent by each node in units of *Mbps*. The traffic distribution designed in this way conforms to the network traffic distribution rules at different times of day and can accurately restore the real environment. We wrote Python scripts to collect network status information at regular intervals and stored it as pickle format files. We collected network status data for 120 moments for each topology. To facilitate verification of the validity of the experiment, the files with the three topologies were both in.xml format and the data of the generated pickle format files were uploaded to the open source platform GitHub.¹

A. Parameter Settings

The DHRL-FNMR agent consists of two layers of controllers, and the neural network structures of the intrinsic controller and meta-controller are the same, consisting of a three-layer convolutional neural network followed by a fully connected output layer. The number of input channels of the first convolutional neural network layer in the intrinsic controller is 5, the number of output channels is 128, the kernel size is (3×3) , the stride is 1, and no padding is performed. The number of input channels of the second convolutional neural network layer is 128, the number of output channels is 256, the kernel size is (3×3) , the stride is 1, and no padding is performed. The number of input channels of the third convolutional neural network layer is 256, the number of output channels is 128, the kernel size is (3×3) , the stride is 1, and no padding is performed. The number of input channels of the fully connected layer is 8192, and the number of output

¹<https://github.com/GuetYe/DHRL-FNMR>

channels is 7. The Leaky ReLU activation function is applied after each convolutional layer. The number of input channels of the first convolutional neural network layer in the meta-controller is 4, and the number of output channels of the final fully connected layer is 14. The rest of the network structure is the same as that of the intrinsic controller, as shown in the policy network and target network in Fig. 5.

During the experiments, it was found that adjusting the kernel size had little impact on overall convergence, and the numbers of input and output channels had similarly minimal impact, as the input matrix contains many 0 elements, and thus, a complex neural network is not required to complete the feature extraction task.

First, we adjusted the parameters of the intrinsic controller. When doing so, we set the number of destination nodes to one and set the source node as the fork node selected by the meta-controller, making it easy to observe the convergence behavior. In these experiments, we set S12 as the source node and S4 as the destination node. We evaluated the impact of parameter adjustments based on the reward value of the final path $P_{12,4}$ constructed by the agent. More generally, we chose S12 as the source node and S2, S4, and S11 as the destination nodes because we wanted the multicast tree from the source node to the destination node to have multiple paths, and we wanted the agent to face many link selection options when constructing the multicast tree to test the effectiveness of the algorithm.

Through many experiments, we made the following observations. First, when there is a leaf node near the destination node and the overall parameters of the link to the leaf node, such as bandwidth, delay, and packet loss rate, are higher than those of the surrounding nodes, the agent will choose the redundant branch; therefore, we chose S11 as a destination node. Second, consider the situation in which there are multiple paths that can reach a destination node, and the number of steps to reach the destination node on each of these paths is not the same. For example, if S2 is a destination node, we can reach it directly through $e_{(5,2)}$, or we can choose $e_{(5,4)}$ and then reach S2 through $e_{(4,2)}$, or we can choose S4 first and then reach S2 through S6. In this case, the overall parameters of $e_{(5,4)}$ are higher than those of $e_{(5,2)}$ but the overall parameters of $e_{(4,2)}$ are very low, resulting in the overall parameters of the tree as a whole being very low. We chose S2 as a destination node to observe the agent's decision-making process in such a case. Finally, there are seven links connected to S4, meaning that there are multiple paths available to reach this node; therefore, we chose S4 as a destination node to observe whether the agent could construct the optimal multicast tree. The DHRL hyperparameters of the meta-controller were adjusted in the same way as those of the intrinsic controller.

Numerous experimental adjustments were made to the reward function settings, as shown in Fig. 14. In this figure, the elements in each list represent the final reward function value R_g , the single-step reward function value R_{step} , the invalid selection penalty R_{none} , and the loop penalty R_{back} .

It can be seen from the figure that when the ratio of R_g to R_{step} was 1:1, the path constructed by the agent did not converge to the optimal solution, and the path constructed during training was very unstable. This is because the reward

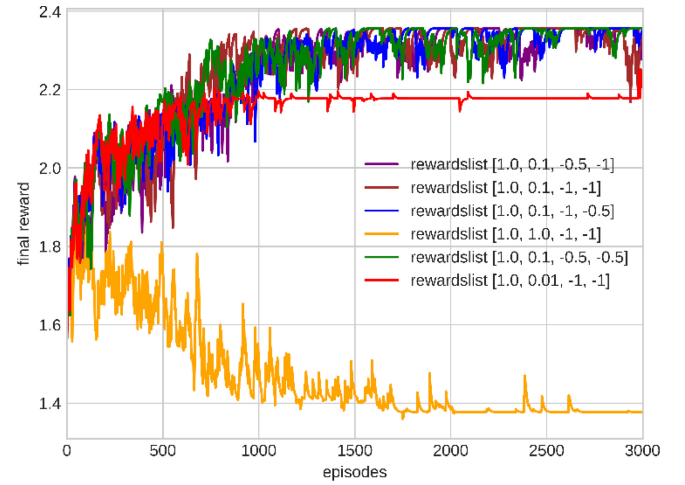


Fig. 14. Reward value adjustment.²

value obtained for each step was very high, and the agent attempted to select more paths to obtain a higher reward value. During this experiment, it was found that the agent would select all paths in the topology because the penalty values were low; even if the agent was punished, it would still choose more paths. Increasing the loop penalty value would only reduce the occurrence of situations in which the agent would walk back along a previously constructed path, and the agent would still construct longer paths.

When the ratio of R_g to R_{step} was 1:0.1, the agent converged to a suboptimal solution. During these experiments, it was found that once the agent reached S5, there were two possible ways to reach S2: either directly through $e_{(5,2)}$ or via $e_{(5,4)}$ followed by $e_{(4,2)}$. The reward value for choosing $e_{(5,4)}$ was higher than that for $e_{(5,2)}$, but the reward value for choosing $e_{(4,2)}$ after reaching S4 was very low. This is because the bandwidth of $e_{(4,2)}$ became the bottleneck bandwidth for the entire path $P_{12,2}$, and the delay and packet loss rate were also relatively high, causing the Q -value for this link to increase after the parameters were updated, and the agent became stuck in a suboptimal solution. In the first few steps of agent decision-making, when the single-step reward is high, it will have a greater impact on decision-making. For example, when starting from S12, the reward value for $e_{(12,14)}$ was greater than that for $e_{(12,13)}$, which caused the agent to always choose $e_{(12,14)}$. However, the reward value for $e_{(14,8)}$ was very small, which affected the parameters for the entire path. In comparison, although the reward value for $e_{(12,13)}$ was slightly smaller, the subsequent reward values were higher than those for $e_{(14,8)}$. Therefore, as seen by considering the above two situations, the single-step reward value should be much smaller than the final reward value to ensure the convergence of the constructed path.

When the ratio of R_g to R_{step} is 1:0.01 and a single-step reward is not needed, the intrinsic controller of the agent can converge to the optimal solution. However, as shown in the figure, when the $R_g : R_{step}$ ratio was 1:0.01, the Q -values

²It is more recognized in the reinforcement learning field that this effect after smoothing converges. Dark colors can highlight the authenticity and originality of the work, but can cause visual ambiguity. In order not to lose the original authenticity of the presentation, please refer to the version provided on arXiv <https://arxiv.org/abs/2305.19077>.

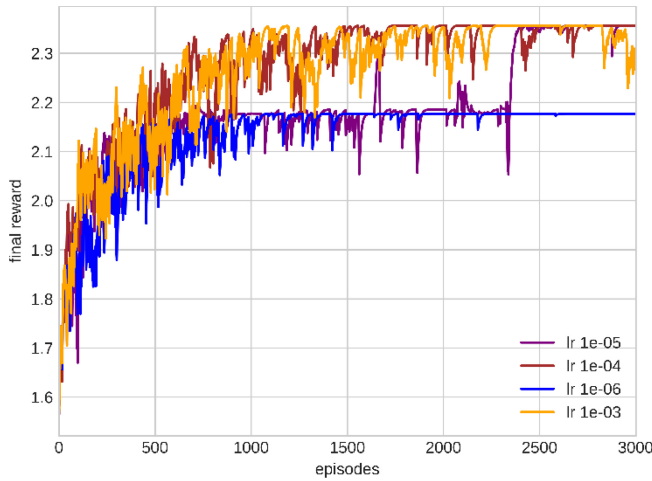


Fig. 15. Learning rate adjustment.

of the link near a destination node were larger, while the Q-values near the source node were very small. Therefore, when the agent was constructing a path, the final reward value would decrease as it was updated backward, leading to the problem of converging to a local optimum.

The penalty values were found to have little effect on overall convergence, although when the penalty values are too large, this can easily lead to unstable convergence. As shown in Fig. 14, when both penalty values were set to -1 , the agent continued to exhibit strong fluctuations after convergence, while when both penalty values were set to -0.5 , the fluctuations were smaller.

Next, the learning rate α was set to different values of 10^{-3} , 10^{-4} , 10^{-5} , and 10^{-6} . In Fig. 15, the vertical axis represents the final reward value of the intrinsic controller. Learning rates of 10^{-3} , 10^{-4} , and 10^{-5} all ensured that the agent could converge to the optimal solution. When α was set to 10^{-3} , the agent converged the fastest, but the later stages of training were very unstable. The curve fluctuated because in the dual network structure, the Q-value of the policy network will approach the Q-value obtained by the target network, and the parameters of the target network will be iteratively updated based on the policy network, causing the loss to be already very low; however, when the parameters of the target network are updated and the loss value becomes very high, this causes the parameters of the policy network to change greatly, leading to deviations in the agent's decision-making. When the learning rate was set to the lowest value of 10^{-6} , the curve converged to a local optimum.

Subsequently, the discount factor γ was set to 0.3, 0.5, 0.7, and 0.9. The discount factor is a multiplicative factor used to reduce the target reward value to be learned. As seen from the final reward values in Fig. 16, when the discount factor is too small, the agent cannot effectively learn from its experience based on the reward value of the current decision. According to the experimental results, a value of 0.9 is the best.

The next step was to set the number of iterations until convergence for the decaying ϵ -greedy method to 100, 500, 1000, and 2000. As shown in Fig. 17, the larger the number of iterations until convergence is, the later the final reward

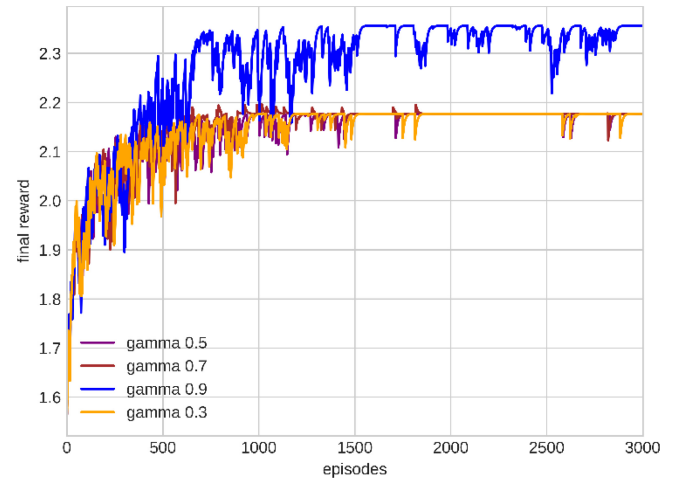


Fig. 16. Discount factor adjustment.

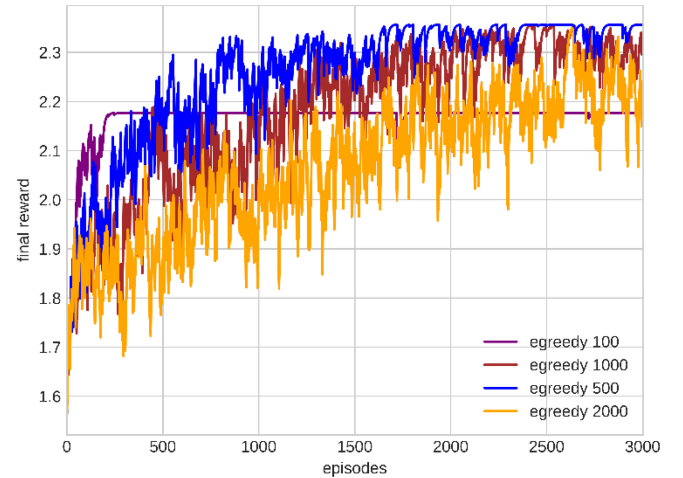


Fig. 17. Adjustment of the convergence epoch.

value converges. When this value is large (1000 or 2000), the agent will perform many explorations and randomly choose actions instead of selecting actions based on their Q-values, which leads to the agent continuously exploring instead of learning from its experience. Although setting this value to 100 caused the agent to converge the fastest, it also lost more exploration opportunities that could have helped it find the optimal solution, leading to convergence to a local optimum. Therefore, to balance the relationship between exploration and exploitation, the number of iterations until convergence was set to 500.

The batch size, which refers to the number of transitions collected by the agent from the experience pool at once, was set to 8, 16, 32, 64, and 128 with the experience pool size fixed at 2048, with the results shown in Fig. 18. Batch sizes of 16, 32, 64, and 128 could all make the agent converge to the optimal path, with little difference between them. When the batch size was set to 8, however, the agent converged to a local optimum. This is because in the early stage of training, the agent encounters many negative examples, such as loops, cycles, and invalid selections, resulting in fewer transitions that successfully contribute to path construction. The PER mechanism ensures the learning efficiency of the agent by

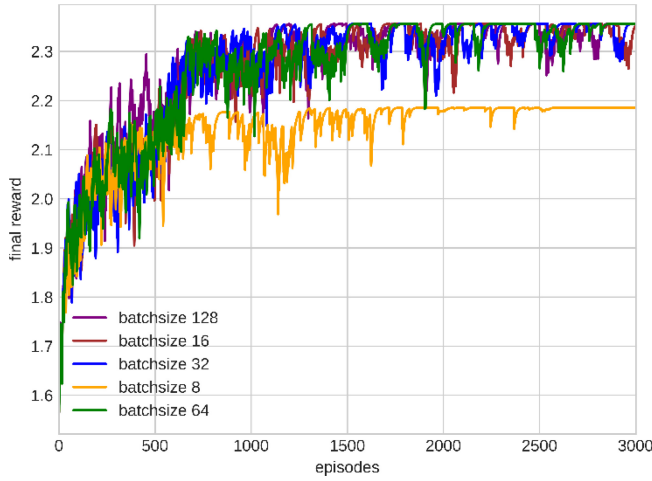


Fig. 18. Adjustment of batch size.

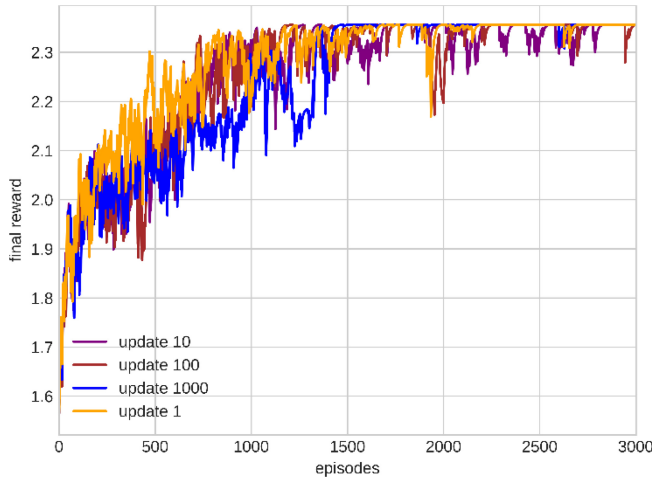


Fig. 19. Adjustment of the update frequency.

means of importance sampling. After the loss function has been calculated for each sample in a batch, the results need to be averaged together, and then the loss is backpropagated. Therefore, when the experience pool contains many positive samples and few negative ones in the later stage of learning, the negative samples will have less impact after averaging, and the agent will converge more stably with a larger batch size. However, the training time will also be longer.

When the experience pool size was changed to 10^4 , it was found that the agent tended to converge to a local optimum with such a large experience pool. When the experience pool is too large, although the agent can potentially discover better transitions, the probability of them being sampled is greatly reduced. In the early stage of exploration, only a few transitions with high reward values will be discovered, and the experience pool will be updated with many negative samples, preventing the agent from effectively utilizing its experience.

Finally, the update interval of the dual network parameters, n_{update} , was set to 1, 10, 100, and 1000, and the results are shown in Fig. 19. When the update interval is too long, as shown by the curve obtained with n_{update} set equal to 100, the agent will converge slowly and exhibit large fluctuations.

In contrast, an excessively high update frequency, as shown by the curve obtained with n_{update} equal to 1, results in frequent small fluctuations. The former is because updating is performed too infrequently for the agent to learn new things in the target network, while the latter behavior occurs because the loss value does not smoothly decrease due to the frequent updates.

B. Comparative Experiments and Performance Analysis

To comprehensively evaluate the overall performance of the proposed algorithm, DHRL-FNMR is compared with five classic or recent SDN multicast methods as far as we know from the current literature, including the near-optimal search algorithms based on greedy search strategies KMB [2], SCTF [44], a heuristic swarm intelligent genetic algorithm GA [45], and two recently proposed routing methods based on reinforcement learning mechanism Q-RL [6], DRL-M4MR [8]. The specific details are as follows:

- **KMB**: Proposed by Kou et al. in 1981 [2], is based on minimum spanning tree and minimum cost paths. In this method, a minimum spanning tree of the complete subgraph containing the destination node is first constructed. It then obtains a subgraph corresponding to the original graph by expanding the edges therein into minimum cost paths, and finally prunes one of the minimum spanning trees of this subgraph to obtain the near-optimal Steiner tree result.
- **SCTF (Selective Closest Terminal First)**: This method [44] finds the solution by designing a link weight change rule and combining it with the shortest path method to add reasonable nodes step by step. It is a method that is widely used in various multicast traffic scheduling scenarios.
- **GA (Genetic Algorithm)**: This method was proposed in 2021 to solve the optimal multicast routing problem for flow scheduling in data center networks [45], [46]. The authors proposed a hybrid genetic algorithm using the k-path individual coding method.
- **Q-RL (Q-learning)**: This method was reported in the literature [6] in 2020 and it mainly solves the unicast routing problem by considering link state information. This paper integrates the k-path approach mentioned in [45] to find the optimal multicast path solution.
- **DRL-M4MR**: Proposed in the literature [8] in 2022, this method entails finding the optimal multicast paths by designing a deep Q network (DQN) reinforcement learning algorithm.

Using the network traffic data collected in each topology scenario described earlier, we implement the above SDN multicast algorithms. For every scenario, we solve every multicast tree by every multicast solution method, and then for every multicast tree, we can calculate the bottleneck bandwidth, delay, and packet loss rate according to Equations (2), (3) and (4), and then find the average value for bottleneck bandwidth, average delay, and average packet loss rate of all the links. Then, these results can be averaged every 10 moments to get the average bottleneck bandwidth, delay,

TABLE I
THE STATISTICAL RESULTS FOR EACH PERFORMANCE INDICATOR

Indicator	Topology	Algorithm	1	2	3	4	5	6	7	8	9	10	11	12	Average
bw	Fat-tree network	DHRL-FNMR	1.21E+01	1.63E+01	1.34E+01	1.18E+01	1.17E+01	7.49E+00	1.38E+01	2.12E+01	1.27E+01	9.89E+00	1.78E+01	1.07E+01	1.32E+01
		DRL-M4MR	1.07E+01	1.63E+01	1.24E+01	1.18E+01	1.00E+01	7.49E+00	1.20E+01	1.86E+01	1.07E+01	9.89E+00	1.78E+01	8.71E+00	1.22E+01
		Q-RL	7.78E+00	1.17E+01	8.70E+00	8.77E+00	5.23E+00	5.05E+00	1.11E+01	1.31E+01	7.19E+00	6.22E+00	1.03E+01	6.51E+00	8.47E+00
		KMB	8.74E+00	1.17E+01	1.04E+01	9.86E+00	8.86E+00	5.37E+00	7.00E+00	1.66E+01	8.66E+00	1.20E+01	1.78E+01	7.21E+00	1.03E+01
		GA	8.28E+00	1.29E+01	1.03E+01	1.06E+01	8.18E+00	7.94E+00	1.32E+01	1.48E+01	6.76E+00	6.80E+00	1.08E+01	6.62E+00	9.77E+00
		SCTF	9.17E+00	1.35E+01	1.13E+01	9.87E+00	8.52E+00	6.81E+00	1.44E+01	1.40E+01	9.05E+00	5.88E+00	1.01E+01	6.61E+00	9.93E+00
	Node14Net	DHRL-FNMR	1.51E+01	1.49E+01	1.40E+01	1.90E+01	9.16E+00	1.24E+01	2.07E+01	1.52E+01	1.01E+01	1.22E+01	1.09E+01	2.07E+01	1.45E+01
		DRL-M4MR	1.40E+01	1.40E+01	1.26E+01	1.89E+01	4.16E+00	3.74E+00	1.10E+01	1.46E+01	5.00E+00	1.08E+01	1.79E+01	3.37E+00	1.08E+01
		Q-RL	4.10E+00	3.68E+00	8.58E+00	7.15E+00	5.03E+00	8.35E+00	2.98E+00	6.13E+00	8.06E+00	4.66E+00	6.68E+00	4.90E+00	5.86E+00
		KMB	1.30E+01	1.40E+01	1.05E+01	9.68E+00	4.16E+00	3.74E+00	1.17E+01	1.50E+01	4.90E+00	1.20E+01	1.29E+01	3.37E+00	9.59E+00
		GA	4.57E+00	4.97E+00	1.03E+01	8.08E+00	5.51E+00	9.81E+00	4.85E+00	6.51E+00	9.84E+00	5.45E+00	7.64E+00	5.70E+00	6.94E+00
		SCTF	9.15E+00	1.13E+01	1.16E+01	1.06E+01	8.60E+00	1.28E+01	1.07E+01	8.03E+00	1.04E+01	1.23E+01	9.12E+00	7.63E+00	1.02E+01
	ARPA-Net	DHRL-FNMR	1.96E+01	1.81E+01	1.81E+01	1.75E+01	1.80E+01	1.53E+01	1.93E+01	2.00E+01	1.72E+01	1.49E+01	1.49E+01	1.46E+01	1.73E+01
		DRL-M4MR	1.62E+01	1.81E+01	1.81E+01	1.75E+01	1.44E+01	1.53E+01	1.60E+01	1.63E+01	1.42E+01	1.49E+01	1.49E+01	1.16E+01	1.56E+01
		Q-RL	1.40E+01	9.02E+00	1.35E+01	1.09E+01	1.08E+01	1.49E+01	1.64E+01	1.12E+01	8.51E+00	1.24E+01	8.23E+00	6.43E+00	1.14E+01
		KMB	1.23E+01	1.23E+01	1.23E+01	1.62E+01	1.10E+01	1.10E+01	1.21E+01	1.21E+01	1.10E+01	1.02E+01	1.02E+01	1.02E+01	1.17E+01
		GA	1.65E+01	1.04E+01	1.44E+01	1.27E+01	1.28E+01	1.62E+01	1.75E+01	1.47E+01	1.22E+01	1.55E+01	1.12E+01	1.06E+01	1.37E+01
		SCTF	1.55E+01	1.08E+01	1.35E+01	1.18E+01	1.22E+01	1.30E+01	1.60E+01	1.28E+01	9.64E+00	1.35E+01	9.30E+00	8.47E+00	1.22E+01
delay	Fat-tree network	DHRL-FNMR	1.30E+01	1.14E+01	1.32E+01	1.28E+01	5.03E+01	4.77E+01	1.26E+01	1.43E+01	1.51E+01	1.32E+01	3.53E+01	1.30E+01	2.10E+01
		DRL-M4MR	6.71E+01	1.14E+01	1.32E+01	1.28E+01	1.14E+01	4.77E+01	8.29E+01	1.27E+01	1.43E+01	1.32E+01	3.53E+01	1.31E+01	2.79E+01
		Q-RL	5.85E+01	2.32E+01	5.15E+01	5.19E+01	7.48E+01	5.18E+01	5.80E+01	4.33E+01	5.77E+01	5.66E+01	4.59E+01	8.53E+01	5.49E+01
		KMB	6.71E+01	1.11E+01	1.32E+01	1.06E+01	1.25E+01	1.95E+02	8.29E+01	1.27E+01	1.43E+01	6.19E+01	3.53E+01	1.31E+01	4.42E+01
		GA	3.46E+01	1.41E+01	2.95E+01	3.74E+01	3.76E+01	2.63E+01	3.46E+01	2.70E+01	3.80E+01	3.32E+01	3.51E+01	5.57E+01	3.36E+01
		SCTF	2.97E+01	1.47E+01	3.27E+01	3.23E+01	2.83E+01	3.34E+01	2.91E+01	2.54E+01	3.47E+01	3.03E+01	3.23E+01	6.29E+01	3.22E+01
	Node14Net	DHRL-FNMR	1.30E+01	1.90E+01	1.16E+01	1.33E+01	5.03E+01	8.94E+00	1.26E+01	1.43E+01	1.51E+01	1.35E+01	1.31E+01	1.30E+01	1.65E+01
		DRL-M4MR	1.29E+01	1.90E+01	1.16E+01	1.33E+01	1.24E+01	8.94E+00	1.26E+01	1.40E+01	1.34E+01	1.35E+01	1.31E+01	1.30E+01	1.31E+01
		Q-RL	3.68E+01	9.08E+01	5.84E+01	3.92E+01	1.68E+01	6.70E+01	1.04E+02	4.20E+01	2.06E+01	6.63E+01	3.11E+01	4.07E+01	5.12E+01
		KMB	1.17E+01	1.90E+01	1.16E+01	1.33E+01	5.88E+01	1.15E+01	1.17E+01	6.37E+01	6.42E+01	2.55E+01	1.32E+01	1.15E+01	2.63E+01
		GA	2.05E+01	2.53E+01	1.94E+01	2.42E+01	1.59E+01	1.26E+01	3.01E+01	1.56E+01	1.23E+01	1.73E+01	1.87E+01	3.26E+01	2.04E+01
		SCTF	1.25E+01	2.36E+01	2.07E+01	2.14E+01	1.22E+01	1.25E+01	1.88E+01	1.50E+01	1.17E+01	1.27E+01	2.79E+01	2.94E+01	1.82E+01
	ARPA-Net	DHRL-FNMR	1.02E+01	1.21E+01	1.21E+01	1.15E+01	8.37E+00	9.26E+00	9.97E+00	1.03E+01	8.22E+00	8.89E+00	8.94E+00	5.63E+00	9.62E+00
		DRL-M4MR	1.36E+01	1.21E+01	1.21E+01	1.15E+01	1.20E+01	9.26E+00	1.33E+01	1.40E+01	1.12E+01	8.89E+00	8.94E+00	8.63E+00	1.13E+01
		Q-RL	1.37E+01	1.45E+01	1.31E+01	1.38E+01	1.31E+01	1.33E+01	1.29E+01	1.37E+01	1.38E+01	1.44E+01	1.41E+01	3.21E+01	1.52E+01
		KMB	1.04E+01	9.96E+00	1.03E+01	9.76E+00	1.17E+01	9.26E+00	1.02E+01	1.01E+01	1.15E+01	7.29E+00	1.20E+01	1.15E+01	1.03E+01
		GA	1.18E+01	1.19E+01	1.14E+01	1.34E+01	1.16E+01	1.22E+01	1.11E+01	1.25E+01	1.18E+01	1.12E+01	1.28E+01	1.26E+01	1.20E+01
		SCTF	1.18E+01	1.34E+01	1.18E+01	1.36E+01	1.20E+01	1.10E+01	1.18E+01	1.30E+01	1.13E+01	1.07E+01	1.25E+01	1.14E+01	1.20E+01

(Continued)

and packet loss rate. As there are a total of 120 moments, we can obtain the 12 group averages. In addition, we also obtain the total average value every 120 moments, such that we get the 13 group statistical averages for all the performance indicators. These results are shown in Table I.

From the above statistics, we can see that: (1) For the performance indicator of average bottleneck bandwidth, although there is a slight decrease in the values of a few individual groups, the method we designed is the best in the final average value for various scenarios with different node sizes and different network topologies. (2) For the performance indicator of average latency, although there is a slight decrease in the 14-node scenario (16.5 ms is only 3 ms more than the best value of 13.1 ms), but it is also the best among the rest of the scenarios. (3) For the packet loss rate performance,

although the performance does not reflect a clear superiority, in fact, all the results are in the order of magnitude of 10^{-4} to 10^{-3} , which is very small and does not reflect a significant difference.

Next, the experimental results for all the scenarios with different topologies are compared and analyzed by means of graphs.

Fat-Tree Network Scenario: The experimental results are shown in Fig. 20. In the average bottleneck bandwidth fold comparison (Fig. 20 a), the blue curve denotes our method, which is at the top of all the other curves, indicating that it is the best. The latency (Fig. 20 b) and packet loss rate (Fig. 20 c) have a couple of performance degradations for some periods, but from the respective absolute value of the vertical coordinate scale, it can be seen that the delay only

TABLE I
(Continued.) THE STATISTICAL RESULTS FOR EACH PERFORMANCE INDICATOR

loss	Fat-tree network	DHRL-FNMR	0.00E+00	0.00E+00	0.00E+00	3.20E-03	3.37E-03	7.17E-04	0.00E+00	1.45E-03	3.74E-04	1.75E-03	1.27E-04	0.00E+00	9.16E-04
		DRL-M4MR	7.56E-04	0.00E+00	0.00E+00	3.20E-03	3.77E-04	7.17E-04	1.26E-03	3.52E-04	0.00E+00	1.75E-03	1.27E-04	0.00E+00	7.12E-04
		Q-RL	8.39E-04	6.87E-05	2.51E-04	1.60E-03	2.55E-04	2.54E-04	7.88E-05	6.24E-04	1.96E-03	1.10E-03	3.81E-03	1.88E-03	1.06E-03
		KMB	7.56E-04	0.00E+00	0.00E+00	2.64E-03	3.77E-04	9.21E-04	1.26E-03	3.52E-04	0.00E+00	1.73E-04	1.27E-04	0.00E+00	5.51E-04
		GA	3.60E-04	1.54E-05	1.49E-04	1.16E-03	6.36E-05	1.84E-04	6.92E-05	5.92E-04	8.08E-04	1.31E-04	3.73E-03	1.70E-03	7.47E-04
		SCTF	3.05E-04	3.58E-05	7.59E-05	2.66E-04	6.65E-05	7.75E-05	1.09E-04	6.57E-04	9.41E-04	2.77E-04	9.31E-04	1.67E-03	4.51E-04
	Node14Net	DHRL-FNMR	0.00E+00	2.50E-03	0.00E+00	0.00E+00	3.37E-03	1.40E-04	0.00E+00	1.45E-03	3.74E-04	0.00E+00	0.00E+00	0.00E+00	6.53E-04
		DRL-M4MR	0.00E+00	2.50E-03	0.00E+00	0.00E+00	0.00E+00	1.40E-04	0.00E+00	1.52E-03	5.34E-04	0.00E+00	0.00E+00	0.00E+00	3.90E-04
		Q-RL	1.61E-03	2.85E-04	1.75E-03	3.14E-04	9.10E-05	1.05E-03	1.08E-03	2.28E-04	1.39E-05	3.57E-04	2.19E-04	1.53E-01	1.33E-02
		KMB	0.00E+00	2.50E-03	0.00E+00	0.00E+00	8.61E-02	1.64E-04	0.00E+00	6.99E-04	0.00E+00	0.00E+00	0.00E+00	0.00E+00	7.46E-03
		GA	1.82E-04	3.28E-05	2.78E-03	1.68E-04	2.38E-05	5.17E-04	1.40E-02	1.29E-04	2.17E-05	2.07E-04	1.87E-04	5.41E-02	6.03E-03
		SCTF	1.16E-04	1.44E-05	0.00E+00	2.75E-04	1.48E-05	2.50E-04	6.19E-04	1.40E-04	2.30E-05	1.49E-04	8.85E-05	7.20E-02	6.14E-03
	ARPA-Net	DHRL-FNMR	0.00E+00	1.30E-03	7.62E-04	0.00E+00	3.37E-03	4.08E-03	0.00E+00	1.45E-03	3.74E-04	4.61E-03	1.11E-03	0.00E+00	1.42E-03
		DRL-M4MR	1.78E-03	1.30E-03	7.62E-04	0.00E+00	3.85E-04	4.08E-03	3.60E-03	3.17E-03	1.12E-03	4.61E-03	1.11E-03	1.66E-03	1.96E-03
		Q-RL	1.45E-04	5.41E-04	5.87E-04	3.04E-04	2.01E-04	7.10E-06	1.53E-04	2.11E-04	2.25E-04	1.41E-03	2.06E-03	3.18E-03	7.51E-04
		KMB	5.99E-02	2.63E-03	2.29E-03	0.00E+00	1.06E-03	5.48E-03	6.34E-03	5.68E-03	3.47E-03	7.85E-02	3.11E-03	3.21E-03	1.43E-02
		GA	6.85E-05	5.05E-04	4.16E-05	7.09E-05	3.76E-04	8.44E-07	1.21E-04	1.30E-04	9.76E-05	4.60E-04	9.38E-04	5.51E-04	2.80E-04
		SCTF	1.09E-04	4.83E-04	7.15E-05	6.59E-05	1.29E-04	3.92E-06	7.68E-05	1.15E-04	7.09E-05	6.95E-04	2.10E-03	2.95E-04	3.52E-04

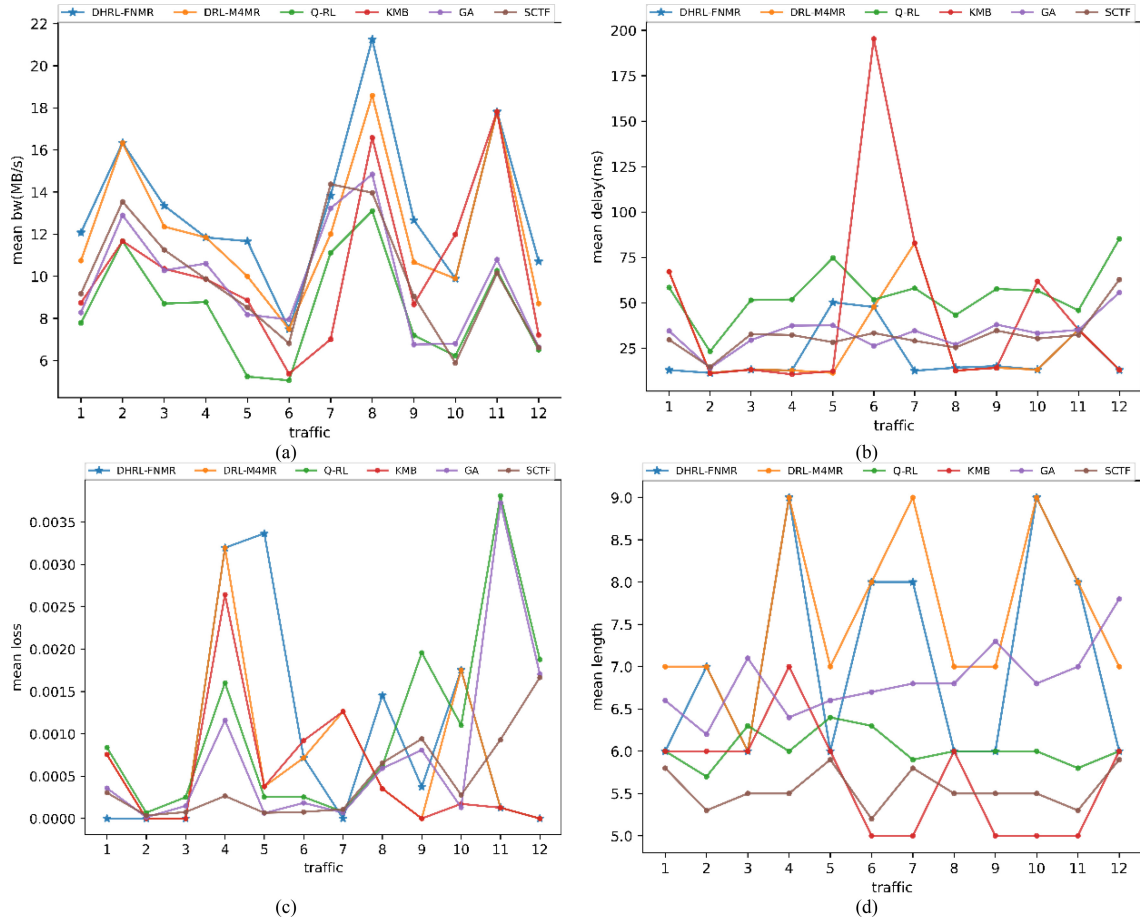


Fig. 20. Performance indicators of each algorithm in Fat-tree network.

has a little decrease in groups 5 and 6, and is still the best for the other groups. Additionally, regarding the comparison curves of the individual packet loss rates, although the ups and downs are large and each method has staggered ups and downs,

the absolute value of the differences between them is within the range of 10^{-4} to 10^{-3} , and the difference in the packet loss rate for such a small order of magnitude is very small. Second, from the optimization of the three objective functions

TABLE II
THE AVERAGE VALUE OF THE PERFORMANCE INDICATORS OF EACH ALGORITHM OF FAT-TREE NETWORK

Algorithm Indicator	DHRL- FNMR	DRL-M4MR	Q-RL	KMB	GA	SCTF
bw	1.32E+01	1.22E+01	8.47E+00	1.03E+01	9.77E+00	9.93E+00
delay	2.10E+01	2.79E+01	5.49E+01	4.42E+01	3.36E+01	3.22E+01
loss	9.16E-04	7.12E-04	1.06E-03	5.51E-04	7.47E-04	4.51E-04

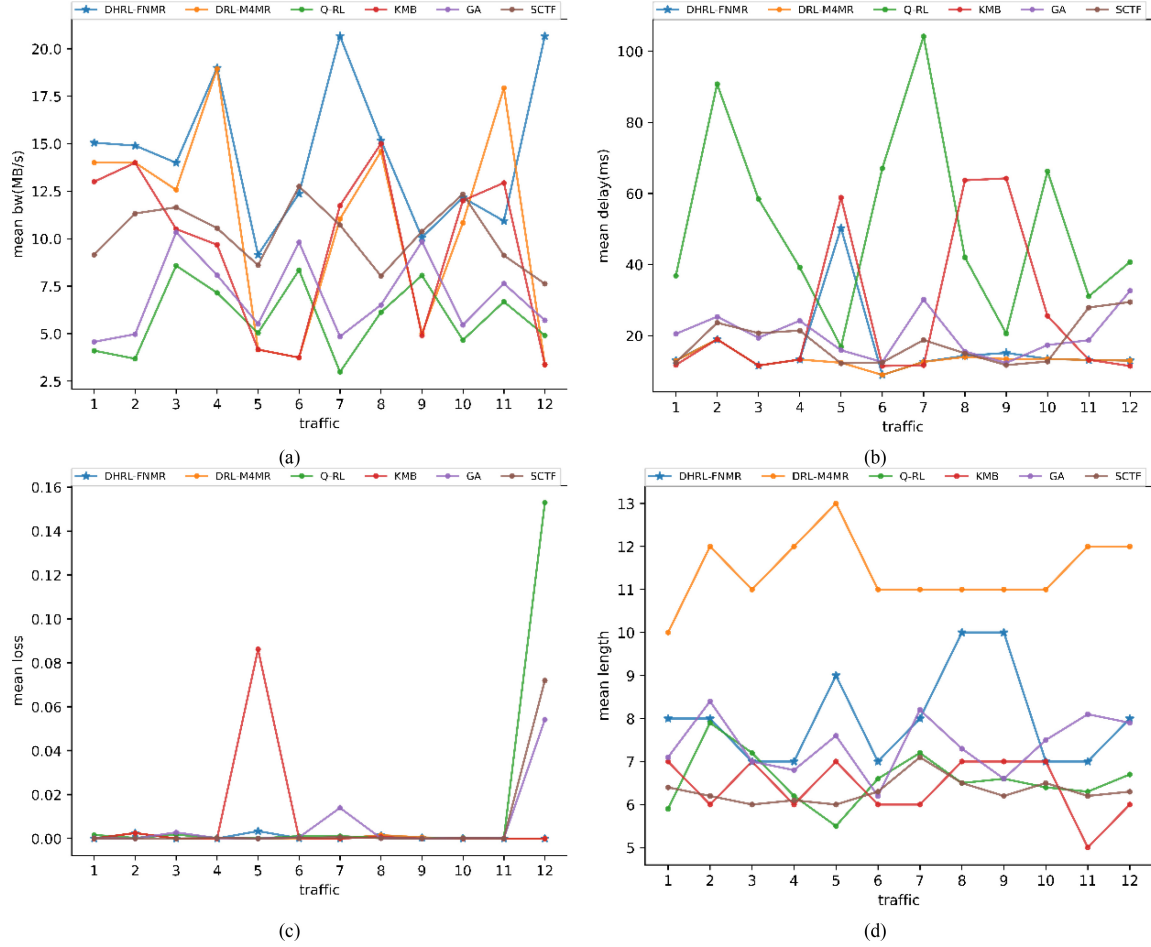


Fig. 21. Performance indicators of each algorithm in Node14Net.

TABLE III
THE AVERAGE VALUE OF THE PERFORMANCE INDICATORS OF EACH ALGORITHM OF NODE14NET

Algorithm Indicator	DHRL- FNMR	DRL-M4MR	Q-RL	KMB	GA	SCTF
bw	1.45E+01	1.08E+01	5.86E+00	9.59E+00	6.94E+00	1.02E+01
delay	1.65E+01	1.31E+01	5.12E+01	2.63E+01	2.04E+01	1.82E+01
loss	6.53E-04	3.90E-04	1.33E-02	7.46E-03	6.03E-03	6.14E-03

Eqs. (2)–(4), the three indicators of bottleneck bandwidth, delay and packet loss rate are not completely independent, and the designed optimization model Eqs. (2)–(4) is essentially a multiobjective optimization problem. Moreover, the values of the three coefficients in the formulas from the design of the incentive functions Eqs. (12)–(14) also show that the optimization model does not result in each indicator appearing to be optimal at the same time. Furthermore, from the statistical averages of all moments (Table II), it can be seen that the method designed in this paper is the best in terms of

bottleneck bandwidth and delay, and in terms of packet loss rate. Although its loss rate is only slightly worse than the rates of the other methods, it can be seen that their differences are in the order of magnitude of 10^{-4} to 10^{-3} , and such an order of magnitude indicates that the difference is very small.

Node14Net Scenario: The experimental results are shown in Fig. 21. Similar to the results obtained in the 10-node scenario, only in the 11th group is the statistical results curve for the average bottleneck bandwidth (Fig. 21 a) not dominant, and the positions of the corresponding blue curves of the rest of

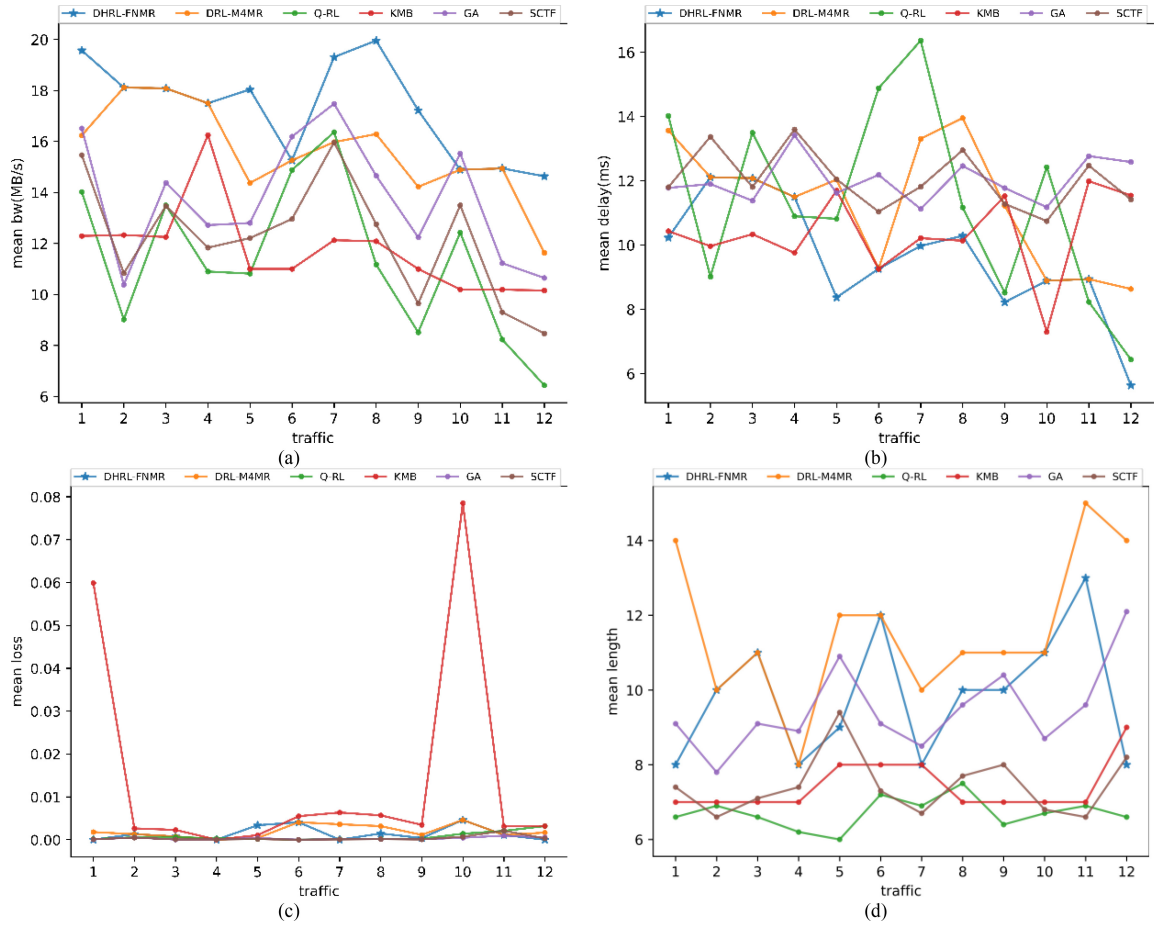


Fig. 22. Performance indicators of each algorithm in ARPA-Net.

TABLE IV
THE AVERAGE VALUE OF THE PERFORMANCE INDICATORS OF EACH ALGORITHM OF ARPA-NET

Algorithm	DHRL-FNMR	DRL-M4MR	Q-RL	KMB	GA	SCTF
Indicator						
bw	1.73E+01	1.56E+01	1.14E+01	1.17E+01	1.37E+01	1.22E+01
delay	9.62E+00	1.13E+01	1.52E+01	1.03E+01	1.20E+01	1.20E+01
loss	1.42E-03	1.96E-03	7.51E-04	1.43E-02	2.80E-04	3.52E-04

the groups are all at the top, which indicates that the method presented in this paper is superior to the other algorithms. The latency (Fig. 21 b) and the packet loss rate (Fig. 21 c) also have a jump in performance at some time points, but we can see that the latency is only reduced in the 5th and 9th groups, and that all other groups can reach the best value. Although there is a small amount of ups and downs, the difference is also the same. The delay (Fig. 21 b) and packet loss rate (Fig. 21 c) also have jumps in performance during some time periods, but it can be seen that the delay is only reduced in groups 5 and 9. All other groups can reach the best value. The packet loss rate curve has a small quantity of ups and downs in group 5, but the difference is also very small. Lastly, from the average of the statistics for all the moments (Table III), the method presented in this paper is the best in the bottleneck bandwidth metrics. Moreover, it is only a little bit worse than the DRL-M4MR in the delay and the packet loss rate metrics and is better than the rest of the algorithms.

ARPA-Net Scenario: The experimental results are shown in Fig. 22. Similar to the above two scenarios, the statistical results curve for the average bottleneck bandwidth (Fig. 22 a) only slightly decreases in the 6th and 10th groups, and the corresponding blue curves for the rest of the groups are all at the top, which indicates that the method presented in this paper is superior to the other algorithms. The latency (Fig. 22 b) and the packet loss rate (Fig. 22 c) have jumpy performance decreases in a few subgroups at certain moments, but when considered with the statistical averages of all moments (Table IV), the method presented in this paper is the best in terms of both average bottleneck bandwidth and latency and in terms of packet loss. Although its value is slightly worse than that of the other methods, it can be seen that it is only in the order of 10^{-3} to 10^{-4} .

From all the above experiments and results, it can be concluded that our proposed DHRL-FNMR algorithm always outperforms other algorithms for the performance metric of

average bottleneck bandwidth. For the performance metric of average delay, although there is a little fluctuation in the performance fluctuation, it still outperforms the other algorithms in terms of the overall reflected effect. For the performance metric of average packet loss rate, although the fluctuation in the performance fluctuation is slightly worse than that of the other algorithms, but similar to the other methods, the average packet loss rate of the proposed methods is in the range of 10^{-3} to 10^{-4} of an order of magnitude.

In addition, it should be emphasized that the three main metrics optimized in this paper are: bandwidth (bw), delay (delay), and packet loss rate (loss). Considering that the length performance metrics of multicast trees are often frequently discussed in the literature, this paper also presents comparative experiments on the length performance metrics of multicast trees. From the results of the average link length comparison experiments for several topologies (Fig. 20 d, Fig. 21 d, Fig. 22 d), our proposed DHRL-FNMR method outperforms the previously worked algorithm DRL-M4MR in all of them, and this result shows that the new algorithm proposed in this paper has been effectively improved our previous algorithm work. In addition, the blue curve corresponding to DHRL-FNMR is slightly higher than those of the other algorithms, which is due to the fact that the reward function in the reinforcement learning algorithm proposed in this paper (Eqs. (12)–(14)) does not include the factor of path length, and does not take the path length parameter as an optimization criterion, but rather only considers the bandwidth, delay, and packet loss rate as the three reasons. Therefore, when constructing a multicast tree, the algorithm designed in this paper will possibly sacrifice the path length to be even larger as a cost in order to prioritize the optimization of these three metrics. This is reasonable, for example, when there is a congested link, in order to reduce the delay increase caused by the congestion, it will choose the path with increased length to send packets. In fact, based on the existing traffic engineering (TM) knowledge, it is sure that if the shortest path is chosen as the route without considering factors such as bandwidth or delay, the criterion of the shortest path will cause the route to become congested and thus reduce the performance of the network. When congestion occurs, it is necessary to complete the route from the longer path appropriately. Therefore, the path length and factors such as delay and bandwidth are sometimes not optimized at the same time.

C. Search Efficiency and Time Complexity Analysis of the Designed HDRL-FNMR Algorithm

A key issue to the effectiveness of the designed reinforcement learning algorithm HDRL-FNMR in solving the multicast routing problem is the need to address the difficulty of slow convergence of the intelligent agent due to the number of redundant branches and too large action space, so this paper also provides experimental statistics on the number of times each node is selected as a fork node by the designed HDRL-FNMR algorithm. Fig. 23 shows the number of times the intelligent agent selects different nodes as fork nodes at the upper-level meta-controller during the training time

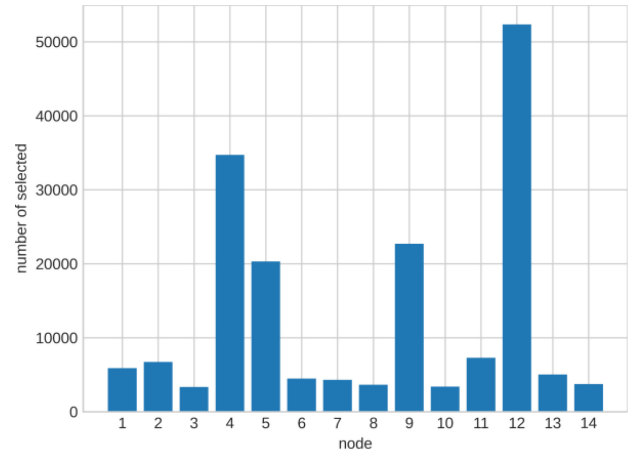


Fig. 23. The number of times each node was selected as a subgoal.

phase of the algorithm when the HDRL-FNMR algorithm is investigated in the scenario represented by the Node14Net topology. Since the meta-controller does a lot of exploration at the beginning, it will try to select every node as a fork node at the beginning of the training, and as the number of training times increases, the meta-controller will select the actions with larger Q-values as fork nodes based on the reward function and the decay ϵ -greedy convergence, and thus it ends up selecting the node 4 and the node 12 as the fork nodes more often. This indicates that with the increase in the number of training times, the search efficiency of the intelligent agent also becomes higher.

A metric that reflects the search efficiency of an algorithm from another perspective is the time complexity. Considering that these algorithms KMB, STFC and GA are not data-driven AI algorithms, they have poor ability to adapt to the scenarios of high-speed dynamic change of network state, and they belong to the near-optimal search algorithms based on the greedy strategy. So the discussion of their time complexity will not be especially comparable, and although the DQL method is a data-driven AI algorithm, the difference between it and the method presented in this paper is that the DQL method has a weaker ability to adapt to the dynamic changes of the network, in addition, it needs to be retrained before each use, which is different from the mechanism of offline training that can be used in the method presented in this paper. Thus, we only carried out experiments to compare the time performance of DRL-M4MR with the same type of DHRL-FNMR method. The time comparison test experiments are conducted in the scenario of Node14Net topology, where the total number of training generations is set to 4000 and the number of Decay ϵ -greedy convergence generations is set to 700, and the average training time of DRL-M4MR is 2 hours and 26 minutes, whereas the average training time of DHRL-FNMR is only 1 hour and 48 minutes. The results for the comparison of the time overhead are shown in Fig. 24. It can be seen that, because the action space design mechanism of the DHRL-FNMR algorithm results in reductions in the intelligent body repeating the selection of invalid action of the negative samples, the search efficiency of the intelligent

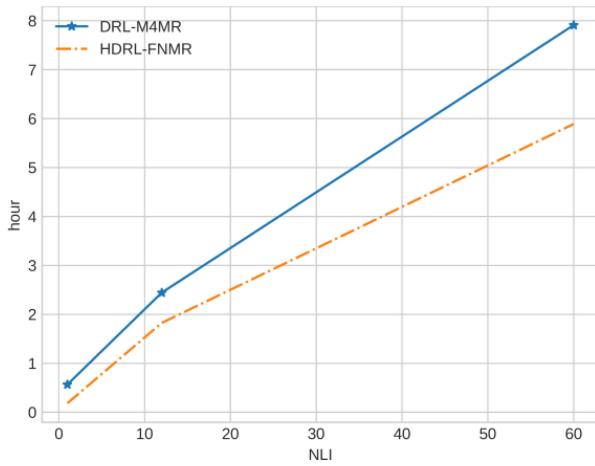


Fig. 24. Time overhead comparison.

body, is effectively improved, thus greatly reducing its learning time.

VII. CONCLUSION

We introduce the DHRL-FNMR algorithm, which leverages the HRL approach to solve the multicast routing problem. The problem is divided into two subproblems: the upper layer is responsible for selecting fork nodes, whereas the lower layer constructs the point-to-point path from a fork node to a destination. We improve upon the DRL-M4MR algorithm by adding a subgoal matrix to the state space of the intrinsic controller. We also design the action spaces for both the intrinsic controller and the meta-controller and propose corresponding methods for finding fork nodes and constructing a multicast tree to the destination nodes. The different actions executed in the action space are categorized and analyzed based on various contexts. Different reward functions are designed for different action situations, and we build on the design of the single-step and final rewards from the DRL-M4MR algorithm. To prevent overestimation during training, we use a dual network. We also use the PER mechanism to balance the exploration and exploitation of the agent.

SDN technology enables the agent to collect NLI and construct a multicast tree with improved bandwidth, decreased delay, and a lower packet loss rate. In contrast to the DRL-M4MR algorithm, the DHRL-FNMR algorithm produces multicast trees without redundant branches. Moreover, the multicast trees generated using the DHRL-FNMR algorithm outperform those generated by the KMB and DRL-M4MR algorithms in terms of average bandwidth, average delay, and average packet loss rate. The agent's search efficiency is also higher when compared to other algorithms. Experimental results demonstrate the ability of the DHRL-FNMR agent to intelligently adjust the optimal multicast paths while flexibly forwarding multicast data in response to changes in the network parameters.

REFERENCES

[1] M. R. Garey, R. L. Graham, and D. S. Johnson, "The complexity of computing Steiner minimal trees," *SIAM J. Appl. Math.*, vol. 32, no. 4, pp. 835–859, 1977.

[2] L. Kou, G. Markowsky, and L. Berman, "A fast algorithm for Steiner trees," *Acta Informatica*, vol. 15, no. 2, pp. 141–145, 1981.

[3] H. Takahashi and A. Matsuyama, "An approximate solution for the Steiner problem in graphs," *Math. Japonica*, vol. 24, no. 6, pp. 573–577, 1980.

[4] A. Y. Hamed, M. H. Alkinani, and M. R. Hassan, "Ant colony optimization for multi-objective multicast routing," *Comput. Mater. Continua*, vol. 63, no. 3, pp. 1159–1173, 2020.

[5] Q. Zhang, L. Ding, and Z. Liao, "A novel genetic algorithm for stable multicast routing in mobile ad hoc networks," *China Commun.*, vol. 16, no. 8, pp. 24–37, Aug. 2019.

[6] D. M. Casas-Velasco, O. M. C. Rendon, and N. L. da Fonseca, "Intelligent routing based on reinforcement learning for software-defined networking," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 1, pp. 870–881, Mar. 2021.

[7] S.-C. Lin, I. F. Akyildiz, P. Wang, and M. Luo, "QoS-aware adaptive routing in multi-layer hierarchical software defined networks: A reinforcement learning approach," in *Proc. IEEE Int. Conf. Services Comput. (SCC)*, 2016, pp. 25–33.

[8] C. Zhao, M. Ye, X. Xue, J. Lv, Q. Jiang, and Y. Wang, "DRL-M4MR: An intelligent multicast routing approach based on DQN deep reinforcement learning in SDN," *Phys. Commun.*, vol. 55, Dec. 2022, Art. no. 101919.

[9] M. Hutsebaut-Buysse, K. Mets, and S. Latré, "Hierarchical reinforcement learning: A survey and open research challenges," *Mach. Learn. Knowl. Extraction*, vol. 4, no. 1, pp. 172–221, 2022.

[10] V. J. Rayward-Smith, "The computation of nearly minimal Steiner trees in graphs," *Int. J. Math. Educ. Sci. Technol.*, vol. 14, no. 1, pp. 15–23, 1983.

[11] S. He, K. Xie, X. Zhou, T. Semong, and J. Wang, "Multi-source reliable multicast routing with QoS constraints of NFV in edge computing," *Electronics*, vol. 8, no. 10, p. 1106, 2019. [Online]. Available: <https://www.mdpi.com/2079-9292/8/10/1106>

[12] Y. Shi, J. Wong, H.-A. Jacobsen, Y. Zhang, and J. Chen, "Topic-oriented bucket-based fast multicast routing in SDN-like publish/subscribe middleware," *IEEE Access*, vol. 8, pp. 89741–89756, 2020, doi: [10.1109/ACCESS.2020.2994268](https://doi.org/10.1109/ACCESS.2020.2994268).

[13] X. Li, Y.-C. Tian, G. Ledwich, Y. Mishra, and C. Zhou, "Minimizing multicast routing delay in multiple multicast trees with shared links for smart grid," *IEEE Trans. Smart Grid*, vol. 10, no. 5, pp. 5427–5435, Sep. 2019, doi: [10.1109/TSG.2018.2882182](https://doi.org/10.1109/TSG.2018.2882182).

[14] G. Bijur, M. Ramakrishna, and K. A. Kotegar, "Multicast tree construction algorithm for dynamic traffic on software defined networks," *Sci. Rep.*, vol. 11, no. 1, 2021, Art. no. 23084.

[15] R.-H. Hwang, M.-C. Peng, C.-Y. Wu, and S. Abimannan, "A novel RPL-based multicast routing mechanism for wireless sensor networks," *Int. J. Ad Hoc Ubiquitous Comput.*, vol. 33, no. 2, pp. 122–131, 2020.

[16] S. Sun, X. Zhang, W. Huang, A. Xu, X. Wang, and L. Han, "QoS-Based multicast routing in network function Virtualization-enabled software-defined mobile edge computing networks," *Mobile Inf. Syst.*, vol. 2021, Apr. 2021, Art. no. 5590963.

[17] A. Sufian, A. Banerjee, and P. Dutta, "Energy and velocity based tree multicast routing in mobile ad-hoc networks," *Wireless Pers. Commun.*, vol. 107, no. 4, pp. 2191–2209, 2019.

[18] S. Kotachi, T. Sato, R. Shinkuma, and E. Oki, "Multicast routing model to minimize number of flow entries in software-defined network," in *Proc. 20th Asia-Pac. Netw. Oper. Manag. Symp. (APNOMS)*, 2019, pp. 1–6.

[19] A. Latif, P. Kathail, S. Vishwarupe, S. Dhesikan, A. Khreishah, and Y. Jararweh, "Multicast optimization for CLOS fabric in media data centers," *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 4, pp. 1855–1868, Dec. 2019.

[20] R. Touihri, S. Alwan, A. Dandoush, N. Aitsaadi, and C. Veillon, "M-CRP: Novel multicast SDN based routing scheme in CamCube server-only datacenter," in *Proc. IEEE Glob. Commun. Conf. (GLOBECOM)*, 2019, pp. 1–6.

[21] C. Risso, F. Robledo, and S. Nesmachnow, "Mixed integer programming formulations for Steiner tree and quality of service multicast tree problems," *Program. Comput. Softw.*, vol. 46, pp. 661–678, Dec. 2020.

[22] S. Shakya and L. N. Pulchowk, "A novel bi-velocity particle swarm optimization scheme for multicast routing problem," *IRO J. Sustain. Wireless Syst.*, vol. 2, no. 1, pp. 50–58, 2020.

[23] S. P. Sahoo and M. R. Kabat, "The multi-constrained multicast routing improved by hybrid bacteria foraging-particle swarm optimization," *Comput. Sci.*, vol. 20, no. 2, pp. 245–269, 2019.

- [24] R. Murugeswari, K. A. Kumar, and S. Alagarsamy, "An improved hybrid discrete PSO with GA for efficient Qos multicast routing," in *Proc. 5th Int. Conf. Electron., Commun. Aerosp. Technol. (ICECA)*, 2021, pp. 609–614.
- [25] X. Zhang, X. Shen, and Z. Yu, "A novel hybrid ant colony optimization for a multicast routing problem," *Algorithms*, vol. 12, no. 1, p. 18, 2019.
- [26] Q. Liu, R. Tang, H. Ren, and Y. Pei, "Optimizing multicast routing tree on application layer via an encoding-free non-dominated sorting genetic algorithm," *Appl. Intell.*, vol. 50, no. 3, pp. 759–777, 2020, doi: 10.1007/s10489-019-01547-9.
- [27] J. Xie et al., "A survey of machine learning techniques applied to software defined networking (SDN): Research issues and challenges," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 1, pp. 393–430, 1st Quart., 2019.
- [28] T. Hendriks, M. Camelo, and S. Latré, "Q 2-routing: A QoS-aware Q-routing algorithm for wireless ad hoc networks," in *Proc. 14th Int. Conf. Wireless Mobile Comput., Netw. Commun. (WiMob)*, 2018, pp. 108–115.
- [29] D. M. Casas-Velasco, O. M. C. Rendon, and N. L. S. da Fonseca, "DRSIR: A deep reinforcement learning approach for routing in software-defined networking," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4807–4820, Dec. 2022.
- [30] X. Guo, H. Lin, Z. Li, and M. Peng, "Deep-reinforcement-learning-based QoS-aware secure routing for SDN-IoT," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6242–6251, Jul. 2020.
- [31] C. Yu, J. Lan, Z. Guo, and Y. Hu, "DROM: Optimizing the routing in software-defined networks with deep reinforcement learning," *IEEE Access*, vol. 6, pp. 64533–64539, 2018.
- [32] G. Kim, Y. Kim, and H. Lim, "Deep reinforcement learning-based routing on software-defined networks," *IEEE Access*, vol. 10, pp. 18121–18133, 2022.
- [33] Z. Ning, N. Wang, and R. Tafazolli, "Deep reinforcement learning for NFV-based service function chaining in multi-service networks," in *Proc. IEEE 21st Int. Conf. High Perform. Switch. Rout. (HPSR)*, 2020, pp. 1–6.
- [34] A. Förster and A. L. Murphy, "Froms: A failure tolerant and mobility enabled multicast routing paradigm with reinforcement learning for WSNs," *Ad Hoc Netw.*, vol. 9, no. 5, pp. 940–965, 2011.
- [35] H.-J. Heo, N. Kim, and B.-D. Lee, "Multicast tree generation technique using reinforcement learning in SDN environments," in *Proc. IEEE SmartWorld, Ubiquitous Intell. Comput., Adv. Trust. Comput., Scalable Comput. Commun., Cloud Big Data Comput., Internet People Smart City Innov. (SmartWorld/SCALCOM/UIC/ATC/CBDCOM/IOP/SCI)*, 2018, pp. 77–81.
- [36] J. Chae and N. Kim, "Multicast tree generation using meta reinforcement learning in SDN-based smart network platforms," *KSII Trans. Internet Inf. Syst.*, vol. 15, no. 9, pp. 3138–3150, 2021.
- [37] T.-N. Tran, T.-V. Nguyen, K. Shim, D. B. Da Costa, and B. An, "A new deep Q-network design for QoS multicast routing in cognitive radio MANETs," *IEEE Access*, vol. 9, pp. 152841–152856, 2021.
- [38] D. Trietsch and F. Hwang, "An improved algorithm for Steiner trees," *SIAM J. Appl. Math.*, vol. 50, no. 1, pp. 244–263, 1990.
- [39] T. D. Kulkarni, K. Narasimhan, A. Saeedi, and J. B. Tenenbaum, "Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation," in *Proc. 30th Adv. Neural Inf. Process. Syst.*, 2016, pp. 1–9.
- [40] L. Liao and V. C. M. Leung, "LLDP based link latency monitoring in software defined networks," in *Proc. 12th Int. Conf. Netw. Service Manag. (CNSM)*, 2016, pp. 330–335.
- [41] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," 2016, *arXiv:1511.05952*.
- [42] Y.-R. Chen, A. Rezapour, W.-G. Tzeng, and S.-C. Tsai, "RL-routing: An SDN routing algorithm based on deep reinforcement learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 3185–3199, Oct.–Dec. 2020.
- [43] "New York metro IBX data center data sheet," Data Sheet, Equinix Internet Serv. Co., Redwood, CA, USA, Accessed: Mar. 14, 2022. [Online]. Available: <https://www.equinix.com/resources/data-sheets/nyc-metro-data-sheet/>
- [44] S. Angelopoulos, "Parameterized analysis of the online priority and node-weighted Steiner tree problems," *Theory Comput. Syst.*, vol. 63, pp. 1413–1447, Apr. 2019.
- [45] W. Ke, Y. Wang, M. Ye, and J. Chen, "A priority-based multicast flow scheduling method for a collaborative edge storage datacenter network," *IEEE Access*, vol. 9, pp. 79793–79805, 2021.
- [46] E. Guler, M. Karakus, and F. Ayaz, "Genetic algorithm enabled virtual multicast tree embedding in software-defined networks," *J. Netw. Comput. Appl.*, vol. 209, Jan. 2023, Art. no. 103538.



Miao Ye received the B.S. degree in theoretical physics from Beijing Normal University in 2000, and the Ph.D. degree from the School of Computer Science and Technology, Xidian University in 2006. He is currently a Full Professor and a Ph.D. Supervisor with the Guilin University of Electronic Technology. His research interests include software-defined networking, edge computing and storage, wireless sensor networks, and deep learning.



Chenwei Zhao was born in 1996. He is currently pursuing the Ph.D. degree with the School of Information and Communication, Guilin University of Electronic Technology. His main research interests include reinforcement learning and software defined networking.



Peng Wen was born in 1994. He received the B.S. degree in mathematics and applied mathematics from Yangzhou University, Yangzhou, Jiangsu, China. He received the M.S. degree in probability theory and mathematical statistics from Guangxi Normal University, Guilin, Guangxi, China. He is currently pursuing the Ph.D. degree with the School of Information and Communication, Guilin University of Electronic Technology. His research interests are broadly in the areas of reinforcement learning, software defined networking, stochastic models for communication networks.



Yong Wang was born in 1964. He received the Ph.D. degree from the East China University of Science and Technology, Shanghai, China, in 2005. He is currently a Full Professor and a Ph.D. supervisor with the Guilin University of Electronic Technology. His main research interests are cloud computing, distributed storage systems, software-defined networks, and information security.



Xiaoli Wang received the B.Sc. degree in software engineering in 2008, and the Ph.D. degree in computer science and technology from Xidian University, Xi'an, Shaanxi, China, in 2014, where she is currently an Associate Professor with the School of Computer Science and Technology. Her research interests include large-scale task scheduling, cloud computing, fog computing, and multiobjective optimization.



Hongbing Qiu received the Ph.D. degree in information and communication engineering from Xidian University in 2004. Since 1984, he has been working as a Teacher of Information and Communication Engineering with the Guilin University of Electronic Technology. From February 2012 until August 2012, he was a Visiting Researcher with the University of Minnesota Twin Cities, Minneapolis, MN, USA. He is a Doctoral Supervisor with Xidian University and the Guilin University of Electronic Technology, and the Director of the Ministry of Education Key Laboratory of Cognitive Radio and Information Processing. His major research interests include mobile communication, wireless sensor networks and image signal processing. He is a member of the Communication Theory and Signal Processing Committee of the Communications Institute of China, of the Broadband Wireless IP Standards Working Group, and of the Home Network Standards Working Group.